

SWI — Softwarové inženýrství

Maturitní zápisky · 25 otázek · příprava 15 min, zkoušení 15 min

Obsah

1. Diagramy UML
2. Algoritmus
3. Reprezentace dat
4. Datové typy, proměnné
5. Návrhové vzory
6. Chyby, testování a ladění
7. Šifrování a kódování
8. Kryptosystémy
9. Objektové programování
10. Databáze
11. Normalizace databáze
12. Jazyk SQL
13. Internet
14. Návrh a tvorba obsahového webu
15. Webová stránka
16. CSS kaskáda
17. Vlastnosti CSS
18. Vývoj aplikací v Reactu
19. Webové aplikace
20. Ověřování identity v internetu
21. RESTful
22. ASP.NET
23. Událostmi řízené programování
24. Programovací jazyky
25. Architektura Android aplikací

1 Diagramy UML

💡 Elevator pitch

UML (Unified Modeling Language) je standardizovaný grafický jazyk pro modelování softwaru. Slouží ke komunikaci mezi vývojáři, analytiky a zákazníky — popisuje strukturu i chování systému ještě před tím, než se napíše kód.

► ROZPAD TÉMATU

- **Účel UML:** vizualizace, specifikace, konstrukce a dokumentace softwarového systému (OMG standard).
- **Dvě hlavní skupiny diagramů:**
 - **Strukturální** (statika — z čeho se systém skládá): třídový (class), objektový, komponentový, nasazení (deployment), balíčkový (package).
 - **Behaviorální** (dynamika — jak systém funguje v čase): use-case, sekvenční, aktivitní, stavový, komunikační, časový.
- **Class diagram** — třídy + atributy + metody + vztahy. Vztahy:
 - **Asociace** (čára) — třídy spolu komunikují.
 - **Agregace** (prázdný kosočtverec) — „má“ vztah, část přežije celek.
 - **Kompozice** (plný kosočtverec) — silné „má“, část zaniká s celkem.
 - **Dědičnost** (prázdná šipka) — generalizace.
 - **Implementace** (přerušovaná šipka) — třída implementuje rozhraní.
- **Multiplicita:** 1, 0..1, *, 1..*, n..m.
- **Viditelnost atributů:** + public, - private, # protected, ~ package.
- **Use-case diagram:** aktér (panáček) + případ užití (ovál) + hranice systému. Vztahy <<include>> (povinné rozšíření) a <<extend>> (volitelné).
- **Sekvenční diagram:** osa Y = čas, šipky mezi „lifelines“ objektů — modeluje pořadí volání metod.
- **Schémata pro databáze:** nejde o klasické UML, ale často se používá **ER diagram** (entita-vztah) nebo class diagram s anotacemi sloupců.

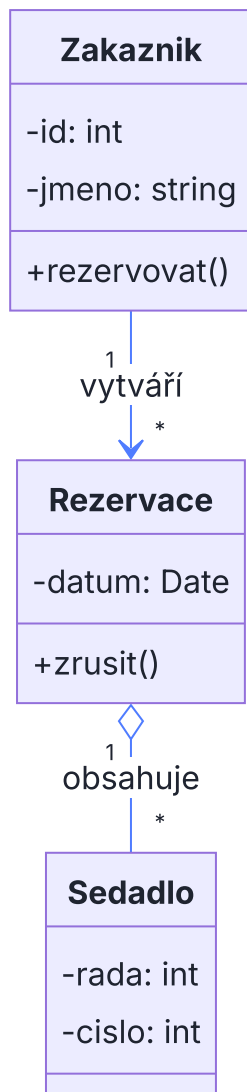
► SROVNÁNÍ DIAGRAMŮ

Diagram	Co modeluje	Typický scénář použití
Class	Statická struktura tříd a vztahů	Návrh objektového modelu před kódem
Use-case	Funkční požadavky z pohledu uživatele	Sběr požadavků se zákazníkem
Sekvenční	Časová posloupnost zpráv	Popis scénáře přihlášení, objednávky
Aktivitní	Tok řízení / workflow	Business proces, algoritmus
Stavový	Stavy objektu a přechody	Životní cyklus objednávky (nová → odeslaná → doručená)
Deployment	HW uzly a běžící komponenty	Rozmístění aplikace na servery

Praktický příklad

Při vývoji rezervačního systému kina nakreslíme nejdřív **use-case diagram** (aktér Zákazník: rezervovat lístek, zrušit rezervaci; aktér Pokladní: tisknout lístek). Z něj odvodíme **class diagram** (třídy `Film`, `Sál`, `Sedadlo`, `Rezervace` s vazbou 1:N) a pro klíčový scénář „rezervace“ doplníme **sekvenční diagram** mezi UI → Controller → `RezervaceService` → DB.

 Co nakreslit na potítku



2 Algoritmus

💡 Elevator pitch

Algoritmus je konečná, jednoznačná posloupnost kroků, která pro daný vstup vede ke správnému výstupu. Je to recept, který musí jít převést do programu.

► VLASTNOSTI ALGORITMU

- **Konečnost** — skončí v rozumném počtu kroků.
- **Determinovanost** — každý krok je jednoznačně definovaný (žádná dvojznačnost).
- **Hromadnost (obecnost)** — řeší celou třídu úloh, ne jen jeden konkrétní vstup.
- **Resultativnost (výstup)** — produkuje výsledek pro každý platný vstup.
- **Korektnost** — výsledek je správný.
- **Elementárnost** — kroky jsou dostatečně jednoduché, aby je vykonal procesor / člověk.

► ZÁPIS ALGORITMU

- **Slovní popis** — přirozený jazyk; rychlý, ale nepřesný.
- **Pseudokód** — formálnější jazyk podobný kódu, ale bez syntaxe konkrétního jazyka.
- **Vývojový diagram** — grafický (obdélník = akce, kosočtverec = větvení, ovál = start/konec).
- **Strukturogram (Nassi-Shneiderman)** — tabulkový blokový zápis.
- **Programovací jazyk** — finální zápis pro počítač.

► ALGORITMICKÁ SLOŽITOST

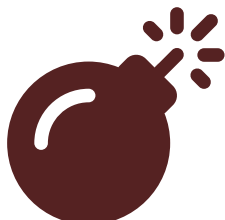
- **Časová** — kolik operací vykoná v závislosti na velikosti vstupu n .
- **Paměťová** — kolik paměti spotřebuje.
- **Asymptotická notace**: O (horní odhad — worst case), Ω (dolní), Θ (těsný).

Třída	Notace	Příklad	$n=1000$ zhruba
Konstantní	$O(1)$	Přístup k poli <code>a[i]</code>	1 op
Logaritmická	$O(\log n)$	Binární vyhledávání	~10
Lineární	$O(n)$	Průchod pole	1 000
Linearitmická	$O(n \log n)$	MergeSort, QuickSort (avg)	~10 000
Kvadratická	$O(n^2)$	BubbleSort, vnořené smyčky	1 000 000
Exponenciální	$O(2^n)$	Naivní hledání podmnožin	nepoužitelné

✖ Praktický příklad

Pro vyhledávání jména v telefonním seznamu (10 000 záznamů) by sekvenční hledání $O(n)$ trvalo až 10 000 porovnání. Pokud seznam seřadíme, binární vyhledávání $O(\log n)$ to zvládne maximálně na ~14 porovnání. Proto databáze udržují **indexy** — aby vyhledávání bylo logaritmické.

✎ Co nakreslit na potítku



Syntax error in text
mermaid version 10.9.5

3 Reprezentace dat

💡 Elevator pitch

Počítač uvnitř pracuje pouze s nulami a jedničkami. Reprezentace dat řeší, jak v této binární formě zaznamenat čísla, znaky, datum nebo strukturované údaje.

► JEDNOTKY INFORMACE

- **Bit (b)** — 0/1, nejmenší jednotka.
- **Bajt (B)** = 8 bitů; 1 znak v ASCII.
- **Předpony:** kB/MB/GB/TB ($10^3 \times$ — síťoví operátoři, disky) vs. KiB/MiB/GiB ($2^{10} \times$ — operační systémy, RAM).

► ČÍSELNÉ SOUSTAVY

Soustava	Základ	Číslice	Použití
Binární	2	0,1	Procesor, paměť
Oktalová	8	0–7	Unix práva (chmod 755)
Dekadická	10	0–9	Lidská
Hexadecimální	16	0–9, A–F	Barvy CSS, paměťové adresy

- **Převod bin → dec:** $1011_2 = 1 \cdot 8 + 0 \cdot 4 + 1 \cdot 2 + 1 \cdot 1 = 11$.
- **Převod dec → bin:** opakované dělení 2 a čtení zbytků zdola nahoru.
- **Záporná čísla:** dvojkový doplněk (negace bitů + 1).
- **Reálná čísla:** IEEE 754 (znaménko, exponent, mantisa) — float (32 b), double (64 b). Pozor na zaokrouhlovací chyby ($0.1 + 0.2 \neq 0.3$ přesně).

► ZNAKY

- **ASCII** — 7 b, 128 znaků (anglická abeceda + řídicí).
- **Windows-1250 / ISO-8859-2** — 8 b, střeoevropská diakritika.
- **Unicode + UTF-8** — variabilní 1–4 B/znak, zpětně kompatibilní s ASCII; dnešní standard webu.

► DATUM A ČAS, SLOŽENÉ TYPY

- **Unix timestamp** — počet sekund od 1.1.1970 UTC (epocha).
- **ISO 8601** — 2026-05-07T14:30:00Z .
- **Základní typy:** int, float/double, char, bool.

- **Složené (strukturované) typy:** pole, struct, třída, záznam, n-tice, výčet (enum).

✂ Praktický příklad

Při návrhu úložiště pro fotky řeším, že barva pixelu se ukládá v hex jako #FF8800 (3×8 b RGB). Časové razítko fotky uložím jako Unix timestamp (8 B int) místo textu — je to úspornější a snadno se podle něj řadí.

✎ Co nakreslit na potítku



4 Datové typy a proměnné

💡 Elevator pitch

Proměnná je pojmenované místo v paměti s určeným datovým typem. Datový typ určuje, jaké hodnoty může držet, kolik zabere paměti a jaké operace nad ní jdou provádět.

► ROZPAD TÉMATU

- **Deklarace vs. inicializace:** deklarace vznáší typ a jméno (`int x;`), inicializace přiřadí hodnotu (`x = 5;` nebo `int x = 5;`).
- **Statická typovost** (C#, Java, C++) — typ se kontroluje při překladu, chyby se chytí brzy.
- **Dynamická typovost** (Python, JS) — typ se určuje za běhu podle hodnoty, je flexibilní ale pomalejší a chyby se odhalí později.
- **Silná vs. slabá typovost** — jak striktně jazyk brání implicitnímu míchání typů.
- **Hodnotové typy** (`int` , `bool` , `struct`) — uloženy přímo na zásobníku, kopírují se hodnotou.
- **Referenční typy** (třídy, pole, string) — na haldě, proměnná drží jen *odkaz*; přiřazení kopíruje odkaz, ne data.
- **Imutabilita** — hodnota se po vytvoření nemění (`string` v C#/Java, `tuple` v Pythonu). Operace vracejí nový objekt.
- **Obor platnosti (scope):** globální, lokální (uvnitř funkce), bloková (uvnitř `{ }`). Vnitřní scope překryje vnější.
- **Přetypování (cast):**
 - **Implicitní** — bezpečné, automaticky (`int` → `long`).
 - **Explicitní** — programátor potvrdí (`(int)3.7`) i s rizikem ztráty.
- **Konstanty** — `const` (kompilační) a `readonly` (běhová, nastavená v konstruktoru).

► HODNOTOVÉ VS. REFERENČNÍ TYPY

Aspekt	Hodnotový	Referenční
Uložení	Stack	Heap (referenec na stacku)
Kopírování	Vznikne nezávislá kopie	Kopíruje se odkaz, sdílí data
Default	0 / false	null
Příklady C#	int, double, struct, enum	class, string, array, delegate

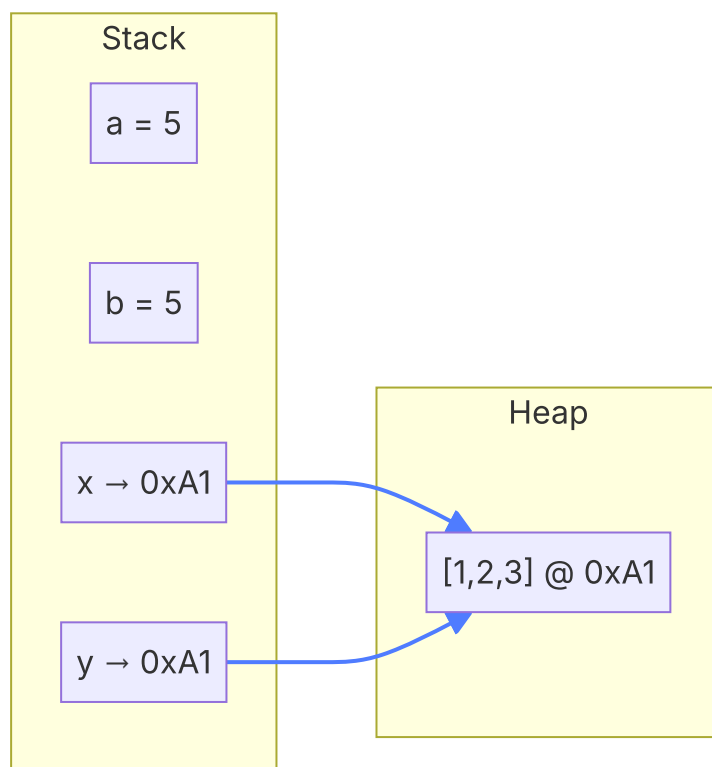
```
int a = 5;
int b = a;    // b je nezávislá kopie
b = 10;      // a pořád 5

int[] x = {1,2,3};
int[] y = x;  // y ukazuje na STEJNÉ pole
y[0] = 99;   // x[0] taky 99 !
```

✂ Praktický příklad

V e-shopu mám třídu `Kosik` (referenční). Když ji předám funkci `SpoctiCenu(kosik)`, funkce dostane odkaz a může košík měnit. Naopak `DPH` (decimal — hodnotový) předávám hodnotou, takže ji uvnitř funkce mohu klidně přepsat bez vlivu na volajícího.

✏ Co nakreslit na potítku



5 Návrhové vzory

💡 Elevator pitch

Návrhový vzor je obecné, opakovaně použitelné řešení často se vyskytujícího problému v návrhu OOP softwaru. Klasifikace pochází od „Gang of Four“ (GoF, 1994) — 23 vzorů ve třech kategoriích.

► TŘI KATEGORIE GOF

Kategorie	Co řeší	Vybrané vzory
Vytvářecí (Creational)	Jak vytvářet objekty	Singleton, Factory Method, Abstract Factory, Builder, Prototype
Strukturální (Structural)	Jak skládat třídy/objekty do větších struktur	Adapter, Decorator, Facade, Composite, Proxy, Bridge, Flyweight
Behaviorální (Behavioral)	Komunikaci a rozdělení odpovědností	Observer, Strategy, Command, Iterator, State, Template Method, Mediator, Memento, Visitor, Chain of Responsibility, Interpreter

► ČASTO ZKOUŠENÉ VZORY V KOSTCE

- **Singleton** — zaručí, že existuje právě 1 instance třídy (privátní konstruktor + statická vlastnost `Instance`). Použití: logger, konfigurace.
- **Factory Method** — místo `new` volám `Create()` , který se sám rozhodne, kterou konkrétní podtřídu vrátí.
- **Builder** — postupné skládání složitého objektu krok po kroku (`builder.SetA().SetB().Build()`).
- **Adapter** — „obal“, který sjednotí cizí API s naším rozhraním (zásuvkový adaptér).
- **Decorator** — dynamicky přidává funkčnost objektu obalením v dalším objektu (`BoldDecorator(text)`).
- **Facade** — jednoduché rozhraní před složitým subsystémem.
- **Observer** — publisher posílá události N přihlášeným subscriberům (event v C#).
- **Strategy** — výměna algoritmu za běhu (`razeni.SetStrategy(quickSort)`).
- **MVC / MVVM** — architektonické vzory oddělující data, prezentaci a logiku.

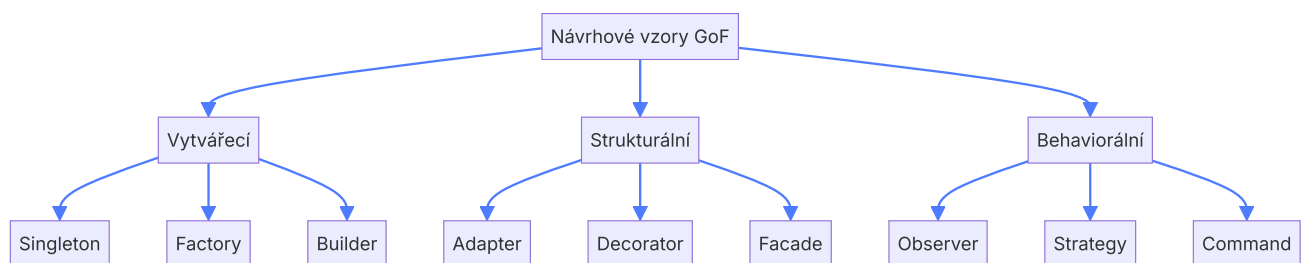
► VÝHODY A NEVÝHODY

Výhody	Nevýhody
Sdílený slovník mezi vývojáři („použij Observer“) Zlepšují udržitelnost a rozšiřitelnost Ověřená řešení — méně chyb v návrhu	Riziko nadužívání („pattern fever“) → zbytečná složitost Vyšší vstupní bariéra pro juniory Mohou skrývat jednoduchá řešení

✂ Praktický příklad

V grafickém editoru chci, aby uživatel mohl přepínat mezi nástroji (štětec, guma, výplň). Všechny mají stejné rozhraní (`IDrawTool`) → použiji **Strategy**: `canvas.SetTool(new BrushTool())` . Pro logování pak globální **Singleton** `Logger.Instance.Log(...)` . A když chci k textovému prvku přidat efekty (tučné, kurzíva, barva), zabalím ho do **Decoratorů**.

✏ Co nakreslit na potítku



6 Chyby, testování a ladění programu

💡 Elevator pitch

Chyby jsou nevyhnutelné. Testování ověřuje, že program dělá to, co má, a ladění (debugging) hledá příčinu chyby, když se objeví. Výjimky jsou mechanismus pro řízené řešení chyb za běhu.

► DRUHÝ CHYB

- **Syntaktické** — porušení gramatiky jazyka. Odhalí překladač před spuštěním.
- **Sémantické / logické** — kód běží, ale dělá něco jiného, než má (špatný vzorec). Nejhorší — překladač je nevidí.
- **Běhové (runtime)** — vznikají za běhu (dělení nulou, přístup mimo pole, `NullReferenceException`).
- **Kompilační** — překlad selže (chybějící typ, nedeklarovaná proměnná).

► VÝJIMKY

- **Throw / try-catch-finally** — výjimka „vybublá“ zásobníkem volání, dokud ji někdo nezachytí.
- **Hierarchie:** všechny dědí z `Exception` (C#) / `Throwable` (Java). Vlastní výjimky vytváříme děděním.
- **Finally** blok se spustí vždy — ideální pro úklid (zavření souboru, spojení).
- **Checked vs. unchecked** (Java) — checked výjimka musí být deklarovaná v signatuře.

```
try {  
    var data = File.ReadAllText("a.txt");  
} catch (FileNotFoundException ex) {  
    Console.WriteLine("Soubor neexistuje: " + ex.Message);  
} catch (Exception ex) {  
    Console.WriteLine("Jiná chyba: " + ex.Message);  
} finally {  
    Console.WriteLine("Vždy proběhne – úklid.");  
}
```

► TESTOVÁNÍ

Úroveň	Co testuje	Nástroje
Unit testy	Jednu metodu/třídu izolovaně	xUnit, NUnit, JUnit, Jest
Integrační	Spolupráci více komponent	+ test DB, mocky API
End-to-end (E2E)	Aplikaci jako celek z pohledu uživatele	Playwright, Selenium, Cypress
Akceptační	Splnění zákaznických požadavků	SpecFlow, Cucumber

- **TDD** (Test Driven Development) — nejdřív test, pak kód: red → green → refactor.
- **Pokrytí (coverage)** — kolik % řádků/větví je pokryto testy.
- **AAA pattern** testu: Arrange (příprav data) → Act (spust operaci) → Assert (ověř výsledek).

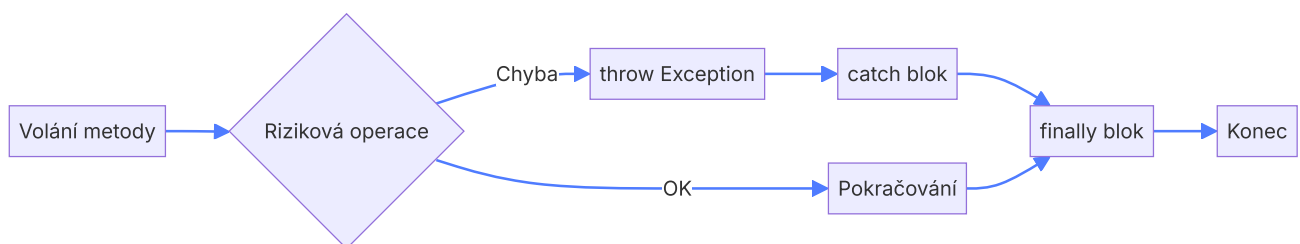
► LADĚNÍ (DEBUGGING)

- **Breakpoint** — bod zastavení; krojujeme F10 (over) / F11 (into).
- **Watch / Locals** — sledování hodnot proměnných.
- **Call stack** — kdo koho volal.
- **Logování** (Serilog, NLog) — záznam za běhu, klíčové v produkci.
- **Profily** — měření výkonu, paměti.

✂ Praktický příklad

V e-shopu padá výpočet ceny. Napíšeme unit test `CenaSDPH_Vstup100_Vraci121()` — selže. Zapneme breakpoint na řádku výpočtu, krojujeme a vidíme, že DPH se násobí dvakrát. Opravíme, test je zelený. V produkci pak chyby logujeme do Serilogu a posíláme do Sentry pro alerting.

✏ Co nakreslit na potítku



7 Šifrování a kódování

💡 Elevator pitch

Kódování převádí data do jiného formátu pro přenos/úložiště (každý ho umí dekódovat). Šifrování chrání obsah před nepovolanými — bez klíče se data nedají rozluštit.

► KÓDOVÁNÍ VS. ŠIFROVÁNÍ

	Kódování	Šifrování
Cíl	Reprezentace dat (přenositelnost)	Utajení dat
Klíč	Není potřeba	Je potřeba
Reverze	Veřejně známá	Pouze s klíčem
Příklady	ASCII, UTF-8, Base64, URL encoding	AES, RSA, Caesar, Vigenère

► KLASICKÉ ŠIFRY

- **Substituční** — Caesar (posun o k), monoalfabetická (libovolná permutace abecedy). Lehce prolomitelné frekvenční analýzou.
- **Polyalfabetická** — Vigenère (klíčové slovo), těžší na frekvenční analýzu.
- **Transpoziční** — písmena se nemění, jen přehazují (skytale).
- **One-time pad** — XOR s klíčem stejné délky jako zpráva, použit jen jednou. Teoreticky nerozluštitelný.

► MODERNÍ SYMETRICKÉ ŠIFRY

- **Symetrické** = stejný klíč pro šifrování i dešifrování. Rychlé, vhodné pro velké objemy dat.
- **Blokové** — šifrují bloky (AES 128/192/256 b, DES — zastaralé, 3DES — překonané).
- **Proudové** — šifrují bit po bitu / byte po byte (ChaCha20, RC4 — nepoužívat).
- **Režimy provozu**: ECB (špatný — opakující se vzory), CBC, CTR, GCM (dnes preferován — i autentizace).

► ASYMETRICKÉ ŠIFRY

- Klíčový pár — **veřejný** (šifruje) a **soukromý** (dešifruje).
- Pomalé, používají se k výměně symetrických klíčů.
- **RSA** (faktorizace velkých čísel), **ECC** (eliptické křivky — kratší klíče, stejná bezpečnost), **Diffie-Hellman** (výměna klíčů přes nezabezpečený kanál).

► HASHOVACÍ FUNKCE

- Jednosměrná funkce — z dat vytvoří otisk pevné délky. Nelze zpětně rekonstruovat.
- **Vlastnosti:** deterministická, lavinový efekt, kolizní odolnost.
- **Algoritmy:** MD5 (prolomený), SHA-1 (slabý), SHA-256, SHA-3, BLAKE2/3.
- **Pro hesla:** bcrypt, Argon2, scrypt, PBKDF2 (pomale záměrně + sůl).

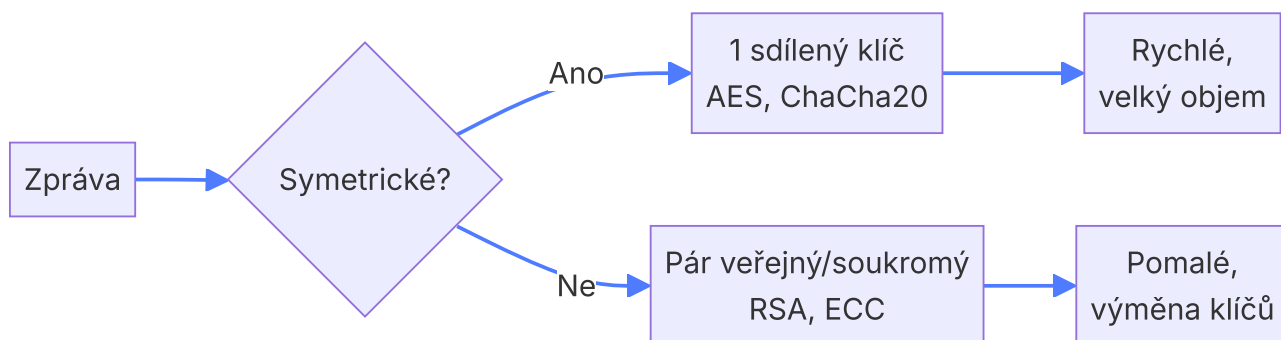
► SLOŽITOST

- Bezpečnost závisí na výpočetní obtížnosti rozluštění bez klíče.
- Délka klíče: 128 b symetricky $\approx$ 3072 b RSA $\approx$ 256 b ECC.
- Hrozba **kvantových počítačů** → post-quantová kryptografie (Kyber, Dilithium).

🔧 Praktický příklad

Když uživatel nahrává soubor do cloudu, klient vygeneruje náhodný AES-256 klíč a soubor jím zašifruje (rychle). Tento AES klíč pak zašifruje veřejným klíčem serveru (RSA). Server si AES klíč rozšifruje svým privátním klíčem a může soubor číst. Tomu se říká **hybridní kryptosystém**.

✏️ Co nakreslit na potítku



8 Kryptosystémy

💡 Elevator pitch

Kryptosystém je ucelený systém pro zabezpečení komunikace — kombinuje šifrování, hashe, digitální podpisy a certifikáty. Cílem je důvěrnost, integrita, autenticita a nepopíratelnost.

► CÍLE BEZPEČNOSTI (CIA + A)

- **Důvěrnost (Confidentiality)** — nikdo nepovoláný nečte data → šifrování.
- **Integrita (Integrity)** — data se cestou nezměnila → hash, MAC.
- **Dostupnost (Availability)** — služba je k dispozici → redundance, DoS ochrana.
- **Autenticita (Authenticity)** — partner je opravdu ten, za koho se vydává → digitální podpis, certifikát.
- **Nepopíratelnost** — odesílatel nemůže popřít, že zprávu poslal → digitální podpis.

► KLÍČOVÉ STAVEBNÍ KAMENY

- **Digitální otisk (hash)** — krátký otisk dat (SHA-256). Slouží k ověření integrity.
- **Digitální podpis** — hash zprávy zašifrovaný *soukromým* klíčem odesílatele. Příjemce jej dešifruje *veřejným* klíčem a porovná s vlastním hashem zprávy.
- **Certifikát X.509** — veřejný klíč podepsaný certifikační autoritou (CA), spolu s identitou držitele.
- **PKI** (Public Key Infrastructure) — strom CA, ve kterém prohlížeč věří kořenovým CA (Let's Encrypt, DigiCert, ...).
- **MAC / HMAC** — hash kombinovaný s klíčem (autentizovaný hash).

► TLS / SSL

- Protokol pro zabezpečenou komunikaci nad TCP. SSL je historický předchůdce, dnes se používá **TLS 1.2 / 1.3**.
- **HTTPS = HTTP + TLS**.
- **Handshake (zjednodušeně, TLS 1.3):**
 1. Klient: ClientHello — podporované šifry + náhodné číslo + svůj veřejný klíč pro DH.
 2. Server: ServerHello — vybraná šifra + svůj klíč pro DH + certifikát + podpis.
 3. Obě strany odvodí stejný symetrický klíč pomocí Diffie-Hellman.
 4. Další komunikace běží symetricky šifrovaná (AES-GCM nebo ChaCha20-Poly1305).
- **Forward secrecy** — i když se v budoucnu prozradí privátní klíč serveru, minulé spojení zůstává tajné (díky efemérnímu DH).

► SYMETRICKÉ VS. ASYMETRICKÉ V KRYPTOSYSTÉMU

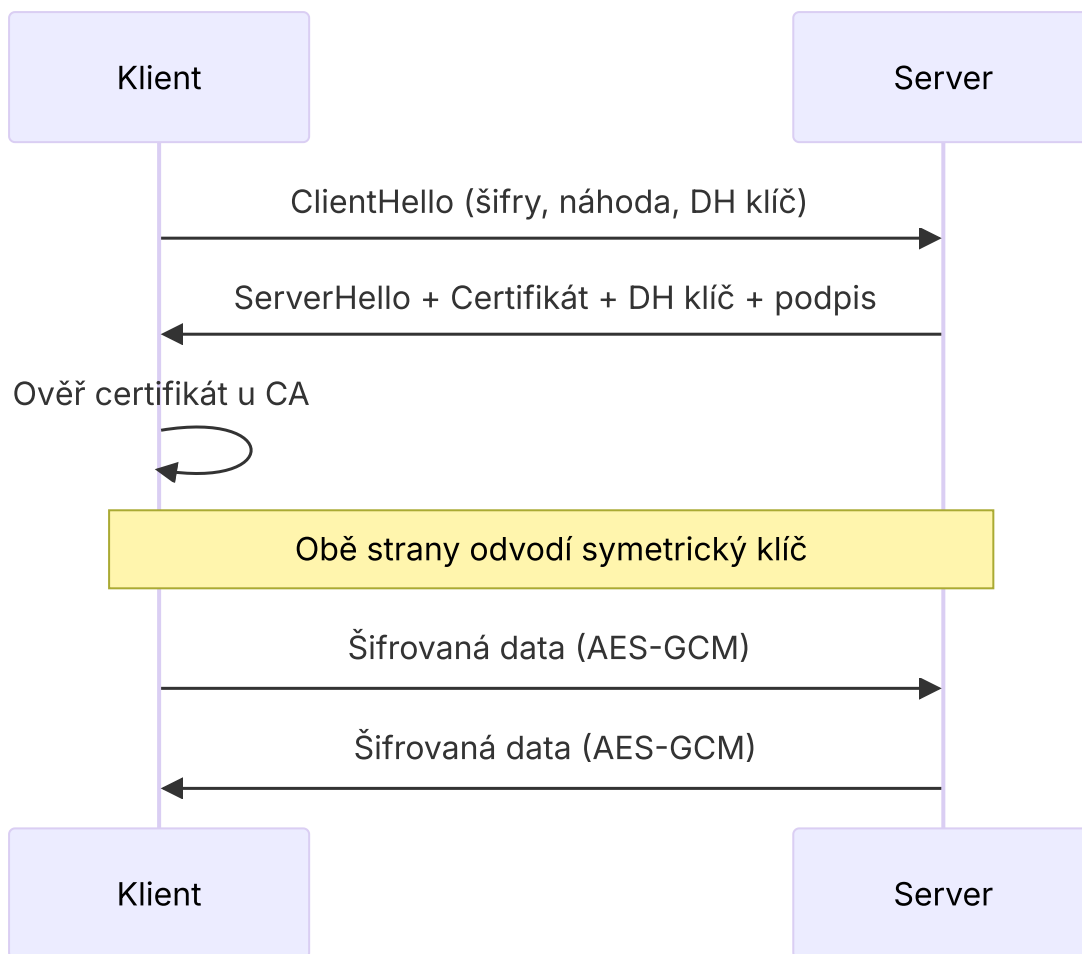
	Symetrická	Asymetrická
Rychlost	Vysoká	~100–1000× pomalejší
Výměna klíče	Problém	Řeší ji
Použití v TLS	Šifrování dat po handshake	Handshake, podpis certifikátu

✳ Praktický příklad

Při návštěvě banky (<https://>) prohlížeč:

1. navazuje TLS spojení, dostane certifikát banky podepsaný kořenovou CA, ověří podpis veřejným klíčem CA;
2. kontroluje doménu a platnost;
3. společně se serverem vygeneruje session klíč;
4. přihlašovací údaje pak putují po síti šifrované AES-GCM — útočník vidí jen zašifrovaný šum.

 Co nakreslit na potítku



9 Objektové programování

💡 Elevator pitch

OOP je paradigma, které modeluje program jako množinu spolupracujících objektů — instancí tříd, jež mají stav (atributy) a chování (metody). Stojí na čtyřech pilířích: zapouzdření, dědičnost, polymorfismus, abstrakce.

► ČTYŘI PILÍŘE OOP

- **Zapouzdření (encapsulation)** — skrytí implementace, přístup přes veřejné rozhraní (vlastnosti, metody). Modifikátory: `private`, `protected`, `internal`, `public`.
- **Dědičnost (inheritance)** — třída přebírá členy rodičovské třídy a může je rozšířit/přepsat. C# má *jednoduchou* dědičnost tříd, ale lze implementovat *více* rozhraní.
- **Polymorfismus** — stejné rozhraní, různá implementace. Forma:
 - **Přetížení (overloading)** — stejný název metody, různé signatury (statický polymorfismus).
 - **Přepsání (overriding)** — potomek poskytne vlastní implementaci virtuální metody (dynamický).
- **Abstrakce** — modelujeme jen podstatné. `abstract` třída nemůže existovat sama, jen jako rodič.

► DALŠÍ KLÍČOVÉ POJMY

- **Třída vs. objekt** — třída je šablona, objekt je instance.
- **Konstruktor / destruktork** — speciální metody pro vznik a zánik objektu.
- **Vlastnosti (properties)** — chovají se jako atribut, ale uvnitř volají getter/setter.
- **Statické členy** — patří třídě, ne instanci (volá se `Math.Sqrt(...)`).
- **Interface (rozhraní)** — kontrakt bez implementace; jen seznam povinných metod/vlastností. Třída může implementovat více rozhraní.
- **Abstraktní třída** — kombinace hotových i abstraktních členů.
- **Generika** — třídy/metody parametrizované typem (`List<T>`) — typová bezpečnost bez duplicity kódu.
- **Jmenné prostory (namespace)** — logické členění, prevence kolize jmen.

► INTERFACE VS. ABSTRAKTNÍ TŘÍDA

Aspekt	Interface	Abstract class
Implementace	Jen kontrakt (od C# 8 i default)	Může mít hotové metody i atributy
Dědění	Lze implementovat více	Jen jedna
Konstruktor	Ne	Ano
Použití	„Co umí" (chování)	„Čím je" (rodičovský typ)

```
public abstract class Zvire {
    public string Jmeno { get; set; }
    public abstract void DejZvuk();    // musí přepsat potomek
    public void Spi() => Console.WriteLine($"{Jmeno} spí.");
}
public class Pes : Zvire, IHracka {
    public override void DejZvuk() => Console.WriteLine("Haf!");
    public void Hraj() => Console.WriteLine("Aportuju.");
}
Zvire z = new Pes { Jmeno = "Rex" };
z.DejZvuk();    // polymorfně volá Pes.DejZvuk → "Haf!"
```

✳️ Praktický příklad

V bankovní aplikaci mám abstraktní třídu `Ucet` s vlastnostmi `Zustatek`, `Cislo` a abstraktní metodou `VypocitejUrok()`. Potomci `SporiciUcet`, `BeznyUcet`, `HypoUcet` mají odlišný výpočet úroku. Interface `IPlatebniMetoda` implementují i třídy, které účet nejsou (`Karta`, `QRPlatba`). Polymorfně pak `foreach (var u in ucty) u.VypocitejUrok();`.

✏️ Co nakreslit na potítku



Syntax error in text
mermaid version 10.9.5

💡 Elevator pitch

Databáze je strukturovaný a perzistentní úložný systém pro data, ke kterému přistupujeme přes systém řízení báze dat (DBMS). Nejrozšířenější je relační model (tabulky + SQL), pro flexibilní data NoSQL.

► ZÁKLADNÍ POJMY

- **DBMS** — software pro správu DB (MySQL, PostgreSQL, MS SQL, Oracle, SQLite, MongoDB).
- **Schéma** — definice tabulek, sloupců, vazeb, omezení.
- **Záznam (řádek, n-tice)** — jeden konkrétní výskyt entity.
- **Atribut (sloupec)** — vlastnost entity s datovým typem.
- **Transakce** — atomická skupina operací s vlastnostmi **ACID**: Atomicity, Consistency, Isolation, Durability.

► KLÍČE

- **Primární klíč (PK)** — jednoznačně identifikuje záznam, NOT NULL, unikátní.
- **Kandidátní klíč** — sloupec/skupina sloupců, která by mohla být PK.
- **Cizí klíč (FK)** — odkaz na PK v jiné tabulce, zajišťuje vazbu.
- **Složený klíč** — PK z více sloupců.
- **Surrogate klíč** — uměle vygenerovaný (auto-increment ID, GUID).

► VAZBY (KARDINALITA)

- **1:1** — Uživatel ↔ Profil (vzácné, často sloučíme do jedné tabulky).
- **1:N** — Zákazník → Objednávky (FK na straně N).
- **N:M** — Studenti × Předměty (řeší se vazební tabulkou).

► INTEGRITA DAT

- **Doménová** — datový typ a constraint sloupce (NOT NULL, CHECK, DEFAULT, UNIQUE).
- **Entitní** — primární klíč nesmí být NULL ani duplicitní.
- **Referenční** — FK musí ukazovat na existující PK; ON DELETE CASCADE / SET NULL / RESTRICT.
- **Uživatelská** — business pravidla (věk > 0, email regexem).

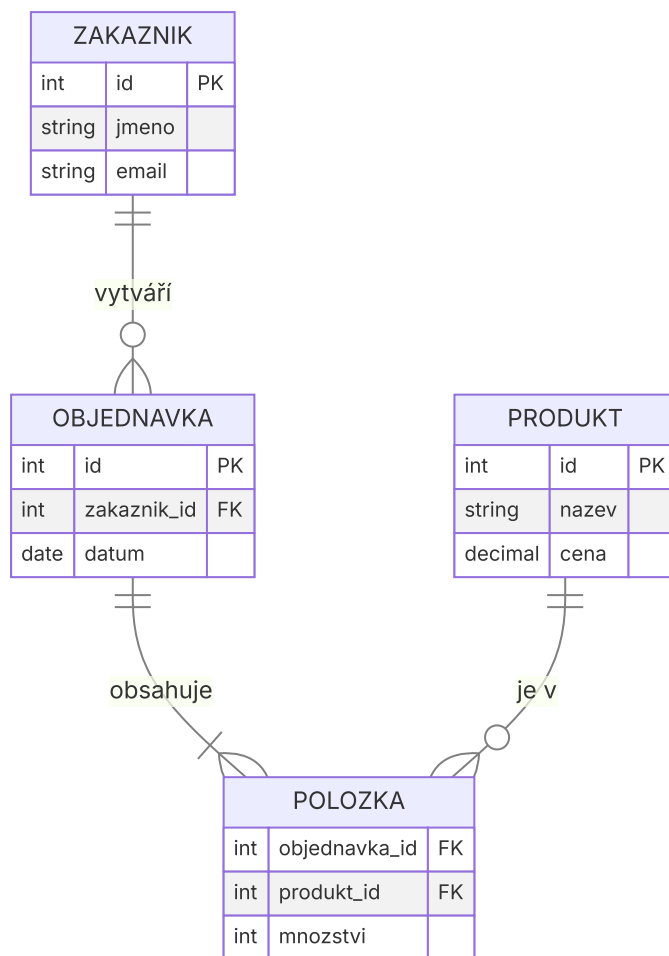
► SQL VS. NOSQL

	Relační (SQL)	NoSQL
Schéma	Pevné, předem dané	Flexibilní, schema-less
Model	Tabulky + vazby	Dokumenty (Mongo), klíč–hodnota (Redis), sloupce (Cassandra), graf (Neo4j)
Dotazy	SQL	API, JSON dotazy
Konzistence	ACID	Často BASE / eventual consistency
Škálovatelnost	Vertikální	Horizontální (sharding)
Použití	Bankovníctví, ERP	Big data, real-time, IoT, sociální sítě

Praktický příklad

V e-shopu zvolím **relační** DB pro objednávky a sklad (transakce, integrita). **Redis** (key-value) pro cache nejprodávanějších produktů a session klíče. **MongoDB** pro recenze, kde každý produkt má jiné atributy a počet hodnocení může být obrovský.

 Co nakreslit na potítku



11 Normalizace databáze

💡 Elevator pitch

Normalizace je proces postupného převedení tabulek do formy, která minimalizuje redundanci a zabraňuje anomáliím při vkládání, mazání a aktualizaci. Postupuje se přes normální formy 1NF → 2NF → 3NF → BCNF.

► ANOMÁLIE V NENORMALIZOVANÉ DATABÁZI

- **Aktualizační** — změna jednoho údaje vyžaduje změnu na více místech.
- **Vkládací** — nelze vložit záznam bez vyplnění nesouvisejících údajů.
- **Mazací** — smazání jednoho záznamu omylem vymaže i jiná data.

► NORMÁLNÍ FORMY

NF	Pravidlo	Co dělat při porušení
0NF	Surová tabulka, opakující se skupiny, vícehodnotová pole	—
1NF	Atomické hodnoty (každá buňka 1 hodnota), jednoznačný PK	Rozdělit vícehodnotové sloupce do nových řádků/tabulek
2NF	1NF + každý neklíčový atribut závisí na <i>celém</i> PK (problém složeného klíče)	Vyčlenit částečné závislosti do nové tabulky
3NF	2NF + žádné <i>tranzitivní</i> závislosti (neklíčový atribut nezávisí na jiném neklíčovém)	Přesunout odvozený atribut do své tabulky
BCNF	3NF + každá funkční závislost má levou stranu jako kandidátní klíč	Přísnější verze 3NF, řeší zbytkové anomálie u složených klíčů

► PŘÍKLAD — OD 0NF K 3NF

```

-- 0NF: jeden řádek = nepořádek
Studenti(jmeno, predmety, ucitel)
"Jan", "Mat;Fyz", "Novák;Svoboda"

-- 1NF: rozdělit
Studenti_Predmety(student, predmet, ucitel)
"Jan", "Mat", "Novák"
"Jan", "Fyz", "Svoboda"

-- 2NF: PK = (student, predmet); ucitel závisí jen na predmet
Studenti(student)
Predmety(predmet, ucitel)
Zapis(student, predmet)

-- 3NF: pokud by Predmety obsahovaly i (predmet, ucitel, kabinet),
-- a kabinet závisel na ucitel (ne na predmet), vyčleníme:
Predmety(predmet, ucitel)
Ucitele(ucitel, kabinet)

```

► OPTIMALIZACE A DENORMALIZACE

- Normalizace = úspora místa + integrita, ale dotaz často potřebuje JOIN y → pomalejší.
- **Denormalizace** — záměrné porušení (zduplikování) pro výkon (datové sklady, cache).

► OLTP VS. OLAP

	OLTP (Online Transaction Processing)	OLAP (Online Analytical Processing)
Účel	Každodenní operace (vklad, objednávka)	Analýza, reporty, BI
Operace	Hlavně INSERT, UPDATE	Hlavně SELECT s agregacemi
Schéma	Normalizované (3NF)	Denormalizované (hvězda, sněhová vločka)
Data	Aktuální, malé objemy / dotaz	Historie, GB–TB
Příklad	Bankovní systém, e-shop	Datový sklad, dashboard tržeb

🔧 Praktický příklad

V e-shopu mám provozní DB v 3NF (objednávky, produkty, kategorie). Pro management vytvářím každou noc **data warehouse** v hvězdicovém schématu — fact tabulka Prodeje + dimenze Cas , Produkt , Region . Reporty pak běží během sekund místo minut.



Co nakreslit na potítku



 Elevator pitch

SQL (Structured Query Language) je deklarativní dotazovací jazyk pro relační databáze — řekne se *co* chceme, ne *jak* to získat. Dělí se do čtyř skupin: DDL, DML, DCL, TCL.

► SKUPINY PŘÍKAZŮ

Skupina	Účel	Příkazy
DDL Data Definition	Definice schématu	CREATE, ALTER, DROP, TRUNCATE
DML Data Manipulation	Práce s daty	SELECT, INSERT, UPDATE, DELETE
DCL Data Control	Oprávnění	GRANT, REVOKE
TCL Transaction Control	Transakce	BEGIN, COMMIT, ROLLBACK, SAVEPOINT

► SELECT — ZÁKLADNÍ STAVEBNÍ KAMENY

```
SELECT sloupec           -- 5. co vrátit
FROM   tabulka           -- 1. odkud
JOIN   jiná ON podmínka  -- 2. spojit
WHERE  filtr_řádků       -- 3. před agregací
GROUP BY sloupec         -- 4. seskupit
HAVING filtr_skupin      -- 6. po agregaci
ORDER BY sloupec [ASC|DESC] -- 7. seřadit
LIMIT n OFFSET m;       -- 8. stránkování
```

► DRUHY JOINŮ

JOIN	Co vrací
INNER JOIN	Pouze řádky se shodou v obou tabulkách
LEFT (OUTER) JOIN	Všechny z levé + shodu z pravé (jinak NULL)
RIGHT JOIN	Všechny z pravé + shodu z levé
FULL OUTER JOIN	Všechny z obou (mimo shodu doplní NULL)
CROSS JOIN	Kartézský součin

► AGREGAČNÍ FUNKCE

- COUNT(*), SUM, AVG, MIN, MAX — vždy nad skupinou definovanou GROUP BY.
- HAVING filtruje skupiny (po agregaci), WHERE řádky (před).

```
-- Top 5 zákazníků podle obrátu v roce 2025
SELECT z.jmeno, SUM(o.castka) AS obrat
FROM   Zakaznik z
JOIN   Objednavka o ON o.zakaznik_id = z.id
WHERE  YEAR(o.datum) = 2025
GROUP BY z.id, z.jmeno
HAVING SUM(o.castka) > 10000
ORDER BY obrat DESC
LIMIT 5;
```

► DDL — SCHÉMA

```
CREATE TABLE Produkt (
  id          INT PRIMARY KEY AUTO_INCREMENT,
  nazev       VARCHAR(100) NOT NULL,
  cena        DECIMAL(10,2) CHECK (cena ≥ 0),
  kategorie_id INT,
  FOREIGN KEY (kategorie_id) REFERENCES Kategorie(id)
  ON DELETE SET NULL
);
ALTER TABLE Produkt ADD COLUMN sklad INT DEFAULT 0;
DROP TABLE Produkt;
```

► INTEGRITNÍ OMEZENÍ

- PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, CHECK, DEFAULT.

► DML — MANIPULACE

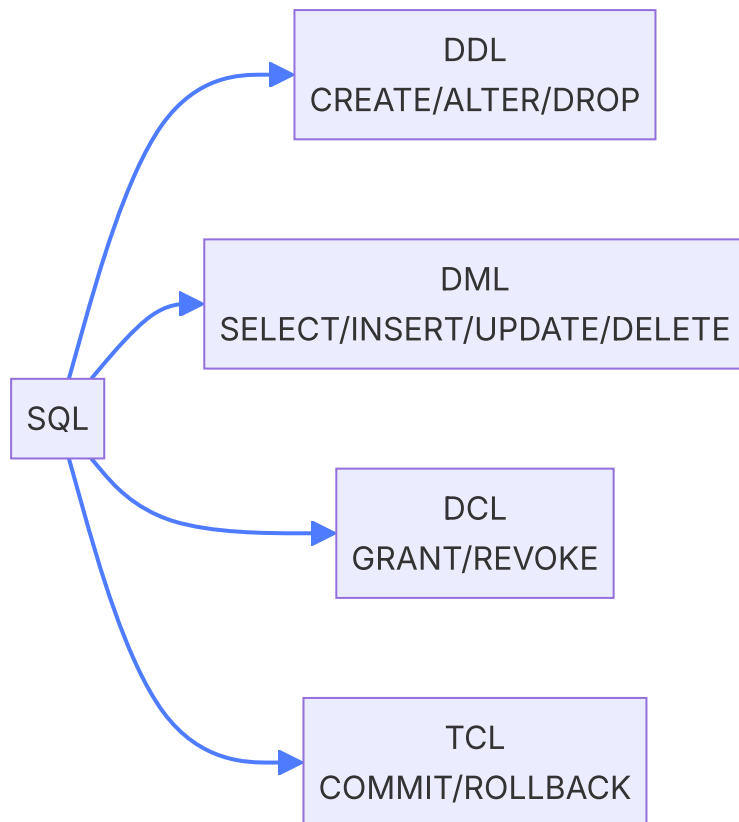
```
INSERT INTO Produkt (nazev, cena) VALUES ('Kniha', 299.00);
UPDATE Produkt SET cena = cena * 1.1 WHERE kategorie_id = 3;
DELETE FROM Produkt WHERE sklad = 0;
```

✳ Praktický příklad

Manažer eshopu chce vědět, ve kterém měsíci roku 2025 vydělal e-shop nejvíc. Napíšu:

```
SELECT MONTH(datum), SUM(castka) FROM Objednavka WHERE YEAR(datum)=2025 GROUP
BY MONTH(datum) ORDER BY 2 DESC LIMIT 1; . Pokud by chtěl jen měsíce s obratem nad
milion, přidám HAVING SUM(castka) > 1000000 .
```

 Co nakreslit na potítku



13 Internet

💡 Elevator pitch

Internet je celosvětová decentralizovaná síť propojených sítí, ve které spolu zařízení komunikují pomocí protokolů TCP/IP. Web (WWW) je jen jedna z mnoha služeb běžících nad ním.

► VRSTVY TCP/IP MODELU

Vrstva	Příklad protokolů	Co řeší
Aplikační	HTTP(S), DNS, FTP, SMTP, IMAP, SSH	Komunikace aplikací
Transportní	TCP, UDP	Doručení mezi procesy (porty)
Síťová	IP, ICMP	Adresace a směrování
Linková	Ethernet, Wi-Fi	Přenos rámců po fyzickém médiu

► IP ADRESY A DOMÉNY

- **IPv4** — 32 b, např. 192.168.1.1 ; ~4,3 mld. adres → vyčerpány.
- **IPv6** — 128 b, např. 2001:db8::1 .
- **Veřejné vs. privátní** — privátní rozsahy (10.0.0.0/8, 192.168.0.0/16) jsou vidět jen v LAN; ven přes **NAT**.
- **DNS** — překlad domény → IP. Hierarchie: kořen → TLD (.cz) → SLD (seznam.cz) → subdoména (www).
- **DNS záznamy**: A (IPv4), AAAA (IPv6), CNAME (alias), MX (mail), TXT (SPF, DKIM), NS (jmenné servery).

► URL — STRUKTURA

`https://www.example.com:443/cesta/soubor.html?q=php&p=2#sekce`

schéma hostname port cesta query fragment

- **Schéma** — protokol (https, http, ftp, mailto).
- **Port** — výchozí 80 (HTTP), 443 (HTTPS), 22 (SSH), 25 (SMTP).
- **Query string** — parametry (?klíč=hodnota&...).
- **Fragment** — kotva v dokumentu (klient ho neposílá serveru).

► ABSOLUTNÍ VS. RELATIVNÍ ADRESA

Absolutní	Relativní
<code>https://web.cz/img/a.png</code>	<code>img/a.png</code> nebo <code>../a.png</code>
Funguje odkudkoliv	Závisí na aktuální URL stránky
Pro externí zdroje	Pro vnitřní odkazy v projektu

► MIME TYPE

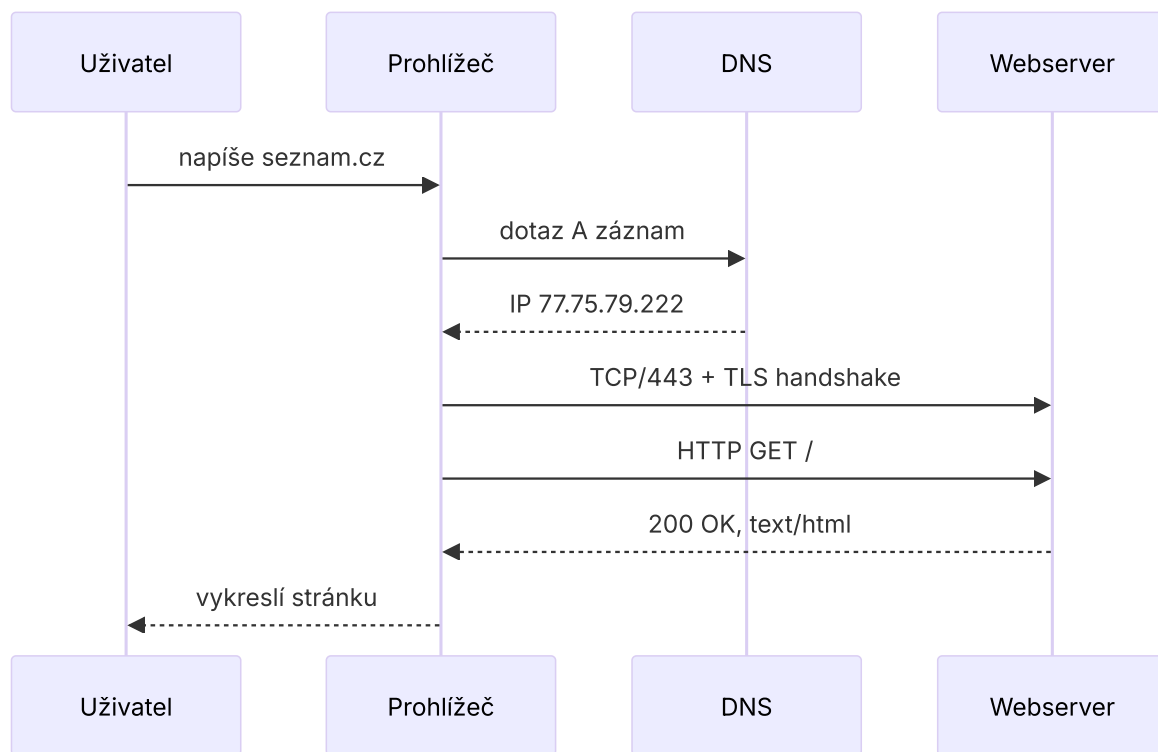
- Standard pro označení formátu obsahu, posílá se v hlavičce `Content-Type`.
- Formát `typ/podtyp`: `text/html`, `application/json`, `image/png`, `application/pdf`, `multipart/form-data`.
- Prohlížeč podle MIME rozhodne, zda obsah zobrazí, stáhne nebo dá pluginu.

Praktický příklad

Když do prohlížeče napíšu `https://seznam.cz`:

1. OS zeptá DNS resolver na IP `seznam.cz` (UDP/53).
2. Naváže TCP spojení na port 443, přes TLS handshake získá session klíč.
3. Pošle HTTP GET, server odpoví `200 OK` s `Content-Type: text/html`.
4. Prohlížeč parsuje HTML, dotahuje CSS, JS a obrázky (každý přes vlastní URL).

 Co nakreslit na potítku



14 Návrh a tvorba obsahového webu

💡 Elevator pitch

Obsahový web (blog, magazín, firemní prezentace) staví na kvalitním textu a přehledné navigaci. Úspěšný projekt vyžaduje promyšlený workflow od analýzy přes UX/UI návrh, vývoj, SEO a po nasazení a měření.

► WORKFLOW TVORBY WEBU

1. **Analýza** — cíl webu, cílová skupina (persona), konkurence, klíčová slova.
2. **Informační architektura** — sitemapa, kategorie obsahu.
3. **Wireframy** — drátěný model rozložení (Figma, Balsamiq).
4. **Vizuální design** — barvy, typografie, ikonografie, brand book.
5. **Prototyp** — klikatelný (Figma).
6. **Implementace** — HTML/CSS/JS, CMS (WordPress, Strapi).
7. **Testování** — UX testy, kompatibilita prohlížečů, výkon.
8. **Nasazení + SEO** — hosting, doména, sitemap.xml, robots.txt.
9. **Měření** — Google Analytics, Search Console; iterace.

► UX VS. UI

UX (User Experience)	UI (User Interface)
Celkový zážitek a použitelnost	Vizuální vrstva — co uživatel vidí
Userflow, persony, A/B testy	Barvy, typografie, ikony, layout
„Funguje to dobře?“	„Vypadá to dobře?“

- **Heuristiky (Nielsen, 10 zásad)**: viditelnost stavu, soulad s reálným světem, kontrola uživatele, konzistence, prevence chyb, rozpoznání místo vzpomínání, ...
- **Above the fold** — co uživatel vidí bez scrollu; nejcennější místo.
- **F-vzor / Z-vzor čtení** — uživatel skenuje text, ne čte slovo po slovu.

► SEO (SEARCH ENGINE OPTIMIZATION)

- **On-page**: klíčová slova, <title> (50–60 zn.), meta description, sémantické HTML, hierarchie nadpisů H1→H6, alt u obrázků, interní prolinkování, čisté URL.
- **Off-page**: zpětné odkazy z důvěryhodných zdrojů.
- **Technické**: rychlost (Core Web Vitals: LCP, INP, CLS), HTTPS, mobilní přívětivost, sitemap.xml, robots.txt, structured data (Schema.org).
- **Obsah** je král — originalita, hloubka, aktualizace.

► RESPONZIVITA

- **Mobile-first** — návrh nejdřív pro malou obrazovku.
- **Viewport** meta tag: `<meta name="viewport" content="width=device-width, initial-scale=1">` .
- **Breakpointy** v CSS media queries (mobil <576, tablet 768, desktop 1024, large 1440).
- **Fluidní jednotky**: %, rem, vw, vh, clamp().
- **Obrázky**: srcset , <picture> , moderní formáty (WebP, AVIF), lazy-loading.

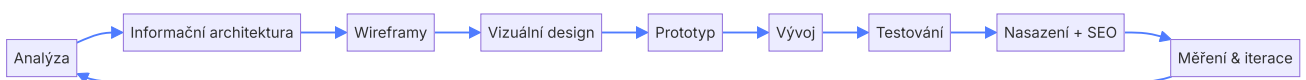
► EFEKTIVITA

- Minifikace CSS/JS, gzip/Brotli kompresi.
- CDN pro statické soubory.
- Cache hlavičky, lazy-loading obrázků a iframů.
- Méně requestů → spritování, ikony jako SVG.

Praktický příklad

Klient chce blog o cestování. Začnu personou („Anna, 28, plánuje dovolenou“), klíčovými slovy v Marketing Mineru, sitemapou (Domů → Destinace → Tipy). Wireframy ve Figmě, schválení, design. Implementace v WordPressu s tématem Astra. SEO: titulky H1 obsahují keyword, alt texty u fotek, structured data Article , sitemap.xml. Po měsíci v Search Console kontroluji pozice a tvořím další obsah.

Co nakreslit na potítku



💡 Elevator pitch

Webová stránka je dokument napsaný v HTML, který prohlížeč po stažení interpretuje a vykreslí. HTML popisuje strukturu a sémantiku obsahu pomocí tagů a atributů.

► ANATOMIE HTML DOKUMENTU

```
<!DOCTYPE html>
<html lang="cs">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Titulek stránky</title>
  <link rel="stylesheet" href="style.css">
  <script src="app.js" defer></script>
</head>
<body>
  <header>...</header>
  <main>...</main>
  <footer>...</footer>
</body>
</html>
```

- **DOCTYPE** — říká prohlížeči, že jde o HTML5.
- **head** — metadata, neviditelný obsah (titulek, CSS, JS, SEO meta).
- **body** — viditelný obsah.

► TAGY — KATEGORIE

Skupina	Tagy	Účel
Strukturální (sémantické)	header, nav, main, section, article, aside, footer	Vyjadřují význam, ne jen vzhled
Textové	h1–h6, p, strong, em, br, hr, blockquote	Text a typografie
Seznamy	ul, ol, li, dl, dt, dd	Odrážky, číslování, definice
Tabulky	table, thead, tbody, tr, th, td	Tabulková data
Multimédia	img, video, audio, picture, source	Obrázky, video, zvuk
Formuláře	form, input, label, select, option, textarea, button	Vstup od uživatele
Inline / generické	span, div, a, code	Bezvýznamový kontejner

► ATRIBUTY

- **Globální** — `id` , `class` , `style` , `title` , `data-*` , `lang` , `hidden` , `tabindex` .
- **Specifické:**
 - `<a href="..." target="_blank" rel="noopener">`
 - `` — `alt` je povinné pro přístupnost.
 - `<input type="text" required minlength="3" placeholder="...">`

► HLAVIČKOVÉ TAGY (HEAD)

- `<title>` — text v záložce a SERP.
- `<meta charset="UTF-8">` — kódování.
- `<meta name="viewport">` — responzivita na mobilech.
- `<meta name="description">` — popis v Googlu.
- `<meta property="og: ...">` — Open Graph (náhled při sdílení na FB, LinkedIn).
- `<link rel="canonical">` — kanonická URL.
- `<link rel="icon">` — favicon.

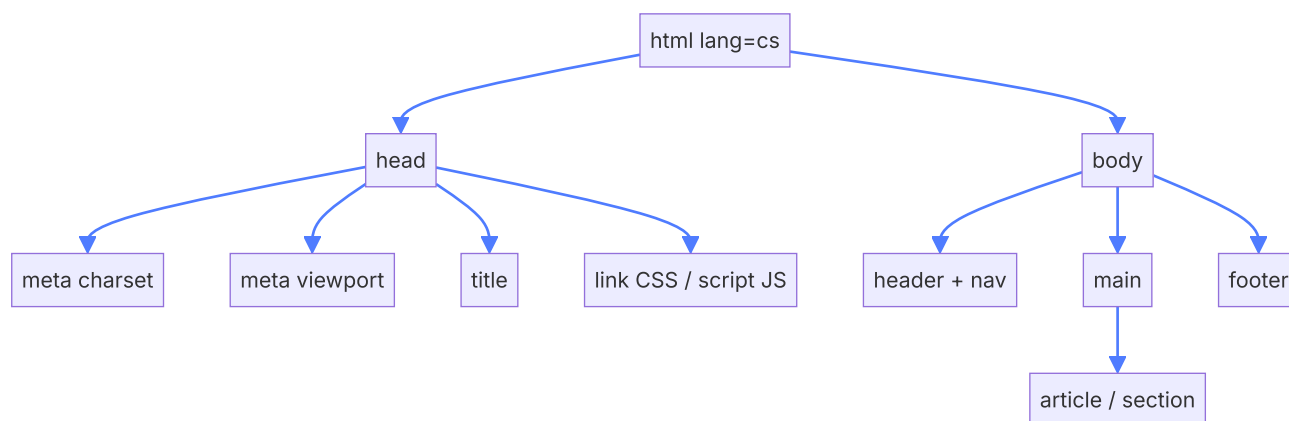
► SÉMANTIKA — PROČ NA NÍ ZÁLEŽÍ

- **SEO** — vyhledávače rozumí struktuře.
- **Přístupnost (WAI-ARIA)** — čtečky obrazovky pro nevidomé.
- **Údržba** — kód je čitelnější.
- Místo „divopolévky“ používáme `<header>` , `<nav>` , `<main>` , `<article>` , `<footer>` .

✳ Praktický příklad

Při tvorbě článku v blogu obalím obsah do `<article>`, do `<header>` dám `<h1>` s nadpisem a `<time>` s datem, postranní informace do `<aside>`. Google si tak snadno vytáhne datum publikace pro rich snippet a screenreader uživateli předčte strukturu („článek, nadpis 1: ...“).

✎ Co nakreslit na potítku



 Elevator pitch

Kaskáda určuje, který CSS deklarace vyhraje, když se na jeden prvek vztahuje více pravidel. Hlavní kritéria jsou: zdroj, specifičnost, pořadí, !important.

► PRAVIDLA KASKÁDY (OD NEJVVYŠŠÍ PRIORITY)

1. **!important** — vždy přebíjí (krom jiného !important s větší specifičností).
2. **Zdroj** a důležitost: user-agent → user → autor → autor !important → user !important → user-agent !important (zjednodušeně).
3. **Inline styl** (atribut `style`) — vysoká specifičnost.
4. **Specifičnost selektoru** — kdo „přesněji“ cílí.
5. **Pořadí** — pozdější deklarace se stejnou specifičností přebije dřívější.

► SPECIFIČNOST

Spočítá se jako čtveřice (inline, ID, třídy/atr/pseudo, prvky/pseudoelementy). Vyšší vlevo > nižší vlevo.

Selektor	Specifičnost
* univerzální	0,0,0,0
p prvek	0,0,0,1
.btn třída	0,0,1,0
#hlavni ID	0,1,0,0
style="..." inline	1,0,0,0
ul li.aktivni	0,0,1,2
:is(h1, h2, h3)	specifita nejvyššího selektoru uvnitř
:where(...)	0 (užitečné pro reset)

► DĚDIČNOST A KASKÁDA

- Některé vlastnosti se dědí (font, color, line-height), jiné ne (padding, border, width).
- `inherit`, `initial`, `unset`, `revert` jsou klíčová slova pro řízení.

► IZOLACE — CASCADE LAYERS, SHADOW DOM

- **@layer** — explicitní pořadí vrstev (`@layer reset, base, components, utilities;`). Mladší vrstvy přebíjejí starší *nezávisle na specifičnosti*.
- **Shadow DOM** — styly uvnitř webové komponenty se nepropisují ven a naopak (kromě CSS proměnných).
- **CSS Modules** (Webpack/Vite) — generují unikátní třídy `.btn_xY3` .

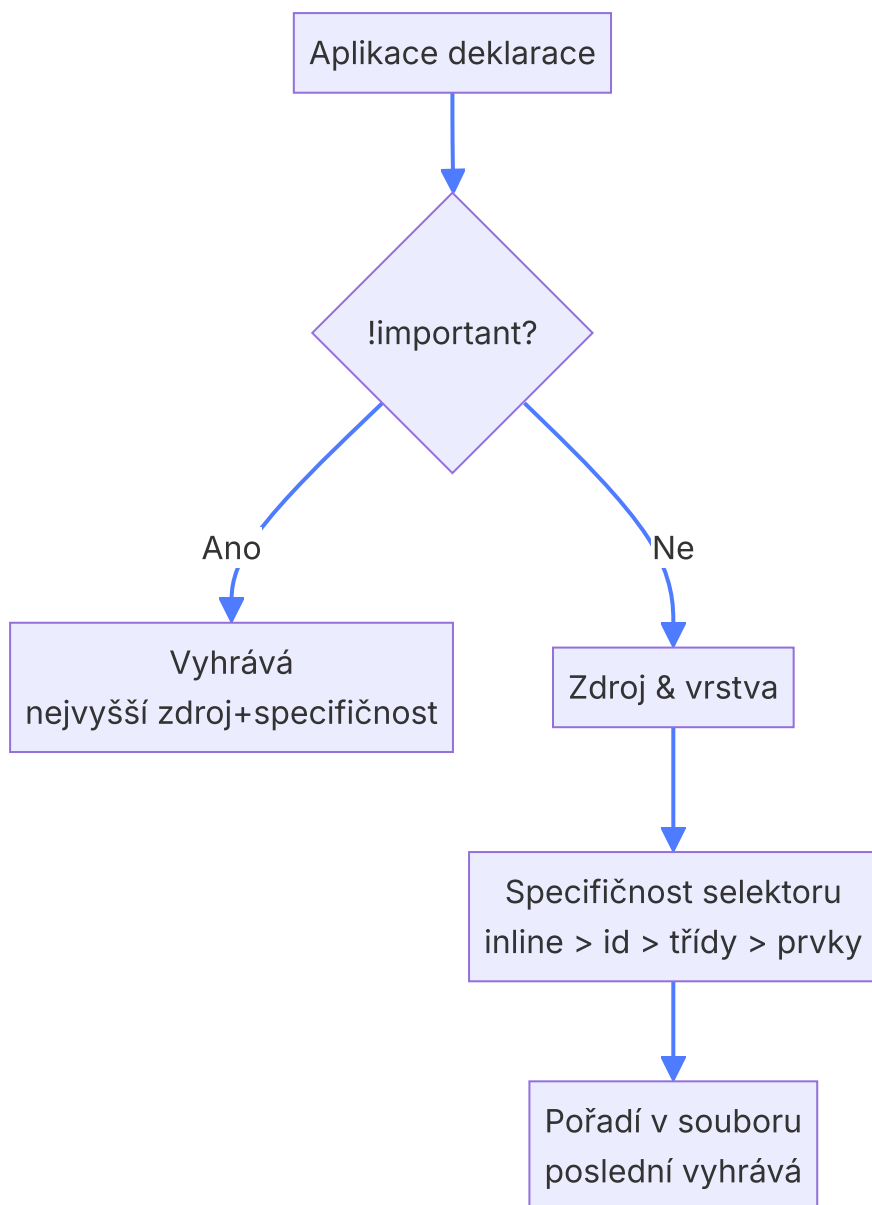
► BEM (BLOCK ELEMENT MODIFIER)

- Konvence pojmenování tříd, která drží specifičnost nízkou a kód předvídatelný.
- **Block** — samostatná komponenta: `.card`
- **Element** — část bloku: `.card__title`
- **Modifier** — varianta: `.card--featured` , `.card__title--big`

```
<article class="card card--featured">
  <h2 class="card__title card__title--big">Akce</h2>
  <p class="card__body">... </p>
</article>
```

✳️ Praktický příklad

V projektu mám konflikt: tlačítko ve formuláři je modré, ale ve footeru má být zelené. Místo přidávání `!important` využiji BEM (`.btn--primary` vs. `.btn--footer`) a vrstvy: `@layer base { .btn { ... } }` `@layer overrides { .footer .btn--footer { ... } }` . Vyhrává `overrides` nezávisle na pořadí v CSS souboru.



💡 Elevator pitch

CSS vlastnosti definují vzhled prvků — barvy, rozměry, písma, mezery, pozici. Hodnoty se zadávají v různých jednotkách a moderní CSS umožňuje proměnné, ligatury a pokročilou typografii.

► JEDNOTKY DÉLKY

Skupina	Jednotky	Vztažené k
Absolutní	px, pt, cm, mm, in	Pevné rozměry; 1in = 96px
Relativní k fontu	em , rem , ex, ch	em = velikost rodiče; rem = root (html)
Relativní k viewportu	vw, vh, vmin, vmax, dvh, svh	1vw = 1 % šířky okna
Relativní k rodiči	%	Záleží na vlastnosti (width, padding...)
Funkce	calc(), min(), max(), clamp()	Kombinace jednotek

```
font-size: clamp(1rem, 1vw + 0.8rem, 1.5rem); /* fluidní typografie */
width: min(100%, 1200px);                      /* nikdy víc než 1200 */
```

► BARVY

- **Pojmenované** — red , aliceblue (140+).
- **HEX** — #ff8800 , zkráceně #f80 ; s alfou #ff8800cc .
- **RGB(A)** — rgb(255 136 0 / 0.8) .
- **HSL(A)** — hsl(30 100% 50%) ; intuitivnější (odstín, sytost, jas).
- **oklch / lab** — moderní, percepčně rovnoměrné.
- `currentColor` — aktuální hodnota `color` .

► CSS PROMĚNNÉ (CUSTOM PROPERTIES)

```

:root {
  --primary: #2a52be;
  --gap: 1rem;
  --radius: 8px;
}
.btn {
  background: var(--primary);
  padding: var(--gap);
  border-radius: var(--radius);
}
.btn:hover { background: color-mix(in oklch, var(--primary), black 20%); }

```

- Kaskádují (dědí se) — můžeme přepsat v podstromu (např. dark mode přes `html[data-theme=dark]`).
- Lze měnit JavaScriptem: `el.style.setProperty('--primary', '#f00')` .

► PÍSMA A TYPOGRAFIE

- **font-family** — fallback stack: `'Inter', system-ui, sans-serif` .
- **@font-face** nebo Google Fonts pro vlastní písmo.
- **font-weight** 100–900, **font-style** italic, **font-size**, **line-height**, **letter-spacing**, **text-transform**.
- **Variabilní fonty** — jeden soubor, plynulé varianty (`font-variation-settings`).

► LIGATURY A OPENTYPE

- **Ligatura** = grafické spojení dvou znaků (např. „fi“ → fi, „fl“).
- V CSS: `font-variant-ligatures: common-ligatures discretionary-ligatures;`
- Další: `font-feature-settings: "tnum" 1, "kern" 1;` (tabulkové číslice, kerning).

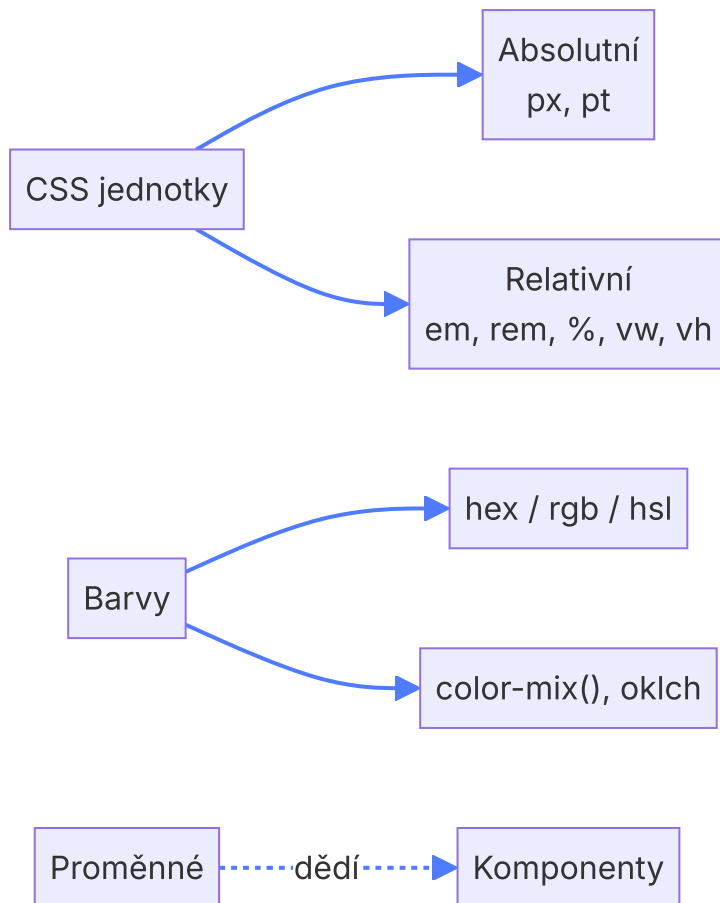
► BOX MODEL

- Content → padding → border → margin.
- `box-sizing: border-box` — width/height zahrne padding+border (čisto).

🔗 Praktický příklad

Pro brand systém definujeme v `:root` proměnné `--brand-primary` , `--brand-success` , `--space-1..6` . Komponenty pak používají `var(--brand-primary)` . Změna značkové barvy = úprava jedné hodnoty. Pro dark mode přidáme `html[data-theme=dark] { --bg: #111; --fg: #eee; }` .

 Co nakreslit na potítku



💡 Elevator pitch

React je deklarativní knihovna od Mety pro tvorbu uživatelských rozhraní. Aplikace se skládá ze znovupoužitelných komponent, které popisují, jak má vypadat UI pro daný stav. Pod kapotou React efektivně aktualizuje DOM přes virtuální DOM.

► DOM A SHADOW DOM

- **DOM** (Document Object Model) — stromová reprezentace HTML, kterou prohlížeč staví v paměti. JS s ní manipuluje (`document.getElementById`).
- **Virtual DOM** — JS objekt, který React drží v paměti. Při změně state se vytvoří nový VDOM a porovná (*reconciliation/diffing*) se starým — DOM se aktualizuje jen tam, kde to je potřeba.
- **Shadow DOM** — zapouzdřený podstrom DOM (Web Components, `<video>`) s izolovanými styly. Nesouvisí přímo s Reactem, ale pojem se zkouší.

► JAVASCRIPT V PROHLÍŽEČI

- JS běží na engine (V8 v Chrome) v jednom vláknu s **event loop**.
- Asynchronní operace přes Web API (`fetch`, `setTimeout`) → zpět do call stacku přes microtask queue (Promise) a task queue.
- ES moduly (`import/export`) podporují všechny moderní prohlížeče.

► JSX — SYNTAXE REACTU

- Rozšíření JS, vypadá jako HTML uvnitř JS.
- Není standardní JS → musí se **transpilovat** (Babel/SWC) na volání `React.createElement(...)` .
- Atributy v camelCase (`className` , `onClick`), výrazy v `{ }` .

```
function Vitej({ jmeno }) {
  const cas = new Date().getHours();
  return (
    <div className="vitez">
      <h1>Ahoj {jmeno}!</h1>
      {cas < 12 ? <p>Dobré ráno</p> : <p>Dobrý den</p>}
    </div>
  );
}
```

► TRANSPILACE A TOOLCHAIN

- **Bundler** — Webpack, Vite, Rollup; spojí moduly do balíčku, vyřeší importy CSS/obrázků.

- **Transpiler** — Babel/SWC převede JSX a moderní JS na verzi srozumitelnou starým prohlížečům.
- **Dev server** s HMR (Hot Module Replacement) — okamžité promítnutí změn bez ztráty stavu.
- **Tree shaking** — odstranění nepoužívaného kódu z bundlu.
- **Lintér (ESLint) + formater (Prettier) + TS** — kvalita kódu.

► FUNKCIONÁLNÍ KOMPONENTY

- Funkce, která vrátí JSX. Stav drží přes **hooks** (`useState` , `useEffect` , `useContext` , `useRef` , `useMemo`).
- Třídní komponenty (`class ... extends Component`) jsou legacy — nové projekty je nepoužívají.
- **Pravidla hooks:** volat na nejvyšší úrovni komponenty, nikdy v podmínce/cyklu, jen v komponentách (nebo dalších hookech).

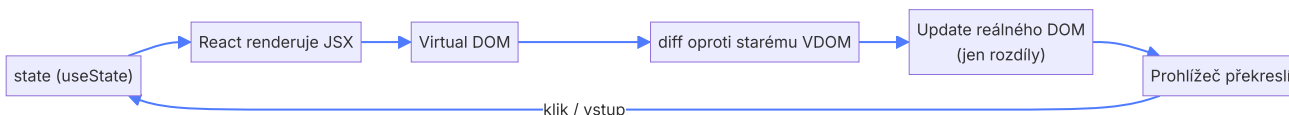
► KLASICKÁ VS. REACT PARADIGMATA

Vanilla JS	React
Imperativní — řekneš jak (<code>querySelector</code> + <code>textContent</code>)	Deklarativní — řekneš co (<code>UI = f(state)</code>)
Manuálně synchronizuješ DOM	Stav → React přerenderuje
Globální stav obtížný	Context, Redux, Zustand

✂ Praktický příklad

Tvořím dashboard tržeb. Vytvořím komponenty `<Filtry>` , `<Graf>` , `<TabulkaProduktu>` . Stav (vybraný měsíc, region) drží rodičovská komponenta a předává dolů přes props. Při změně filtru React přepočítá, který kus DOM se musí změnit, a aktualizuje pouze graf, ne celou stránku. Build dělá Vite s HMR — úprava CSS se ukáže okamžitě bez refrese.

✏ Co nakreslit na potítku



19 Webové aplikace

💡 Elevator pitch

Webová aplikace běží v prohlížeči, ale nabízí interaktivitu klasických desktopových programů. Komunikuje se serverem přes HTTP(S) ve vzoru request–response.

► DRUHY WEBOVÝCH APLIKACÍ

	MPA (Multi-Page Application)	SPA (Single-Page Application)
Princip	Server posílá kompletní HTML pro každou URL	Server pošle 1× shell, dál JS volá API a překresluje
Výhody	Rychlý první render, snadné SEO, méně JS	Plynulé UX bez přenačítání, offline možné
Nevýhody	Při kliku se znovu načte celá stránka	Pomalý first paint, SEO náročnější (řeší SSR)
Technologie	PHP, Razor Pages, WordPress	React, Vue, Angular
Hybrid	SSR/SSG (Next.js, Nuxt) — kombinace výhod obojího	

► REQUEST – RESPONSE

1. Klient (prohlížeč) sestaví HTTP request.
2. Pošle ho přes TCP/TLS na server.
3. Server zpracuje, případně se ptá DB, sestaví response.
4. Klient response vykreslí (HTML, JSON, image...).

HTTP je **bezstavový** — server si nepamatuje předchozí požadavek. Stav se řeší cookies, session, tokeny.

► HTTP VS. HTTPS

- **HTTP** (port 80) — prostý text, lze odposlechnout.
- **HTTPS** (port 443) — HTTP nad TLS — důvěrnost, integrita, autenticita.
- Dnes nutnost (HTTP/2, HSTS, certifikát od Let's Encrypt zdarma).

► HTTP METODY

Metoda	Účel	Idempotentní?	Tělo?
GET	Získat zdroj	Ano	Ne (typicky)
POST	Vytvořit / odeslat data	Ne	Ano
PUT	Nahradit zdroj	Ano	Ano
PATCH	Částečná úprava	Ne (zpravidla)	Ano
DELETE	Smazat	Ano	Ne
HEAD / OPTIONS	Metadata / CORS preflight	Ano	Ne

► STAVOVÉ KÓDY

Třída	Význam	Příklady
1xx	Informační	100 Continue, 101 Switching Protocols
2xx	Úspěch	200 OK, 201 Created, 204 No Content
3xx	Přesměrování	301 Moved Permanently, 302 Found, 304 Not Modified
4xx	Chyba klienta	400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 429 Too Many Requests
5xx	Chyba serveru	500 Internal Server Error, 502 Bad Gateway, 503 Service Unavailable

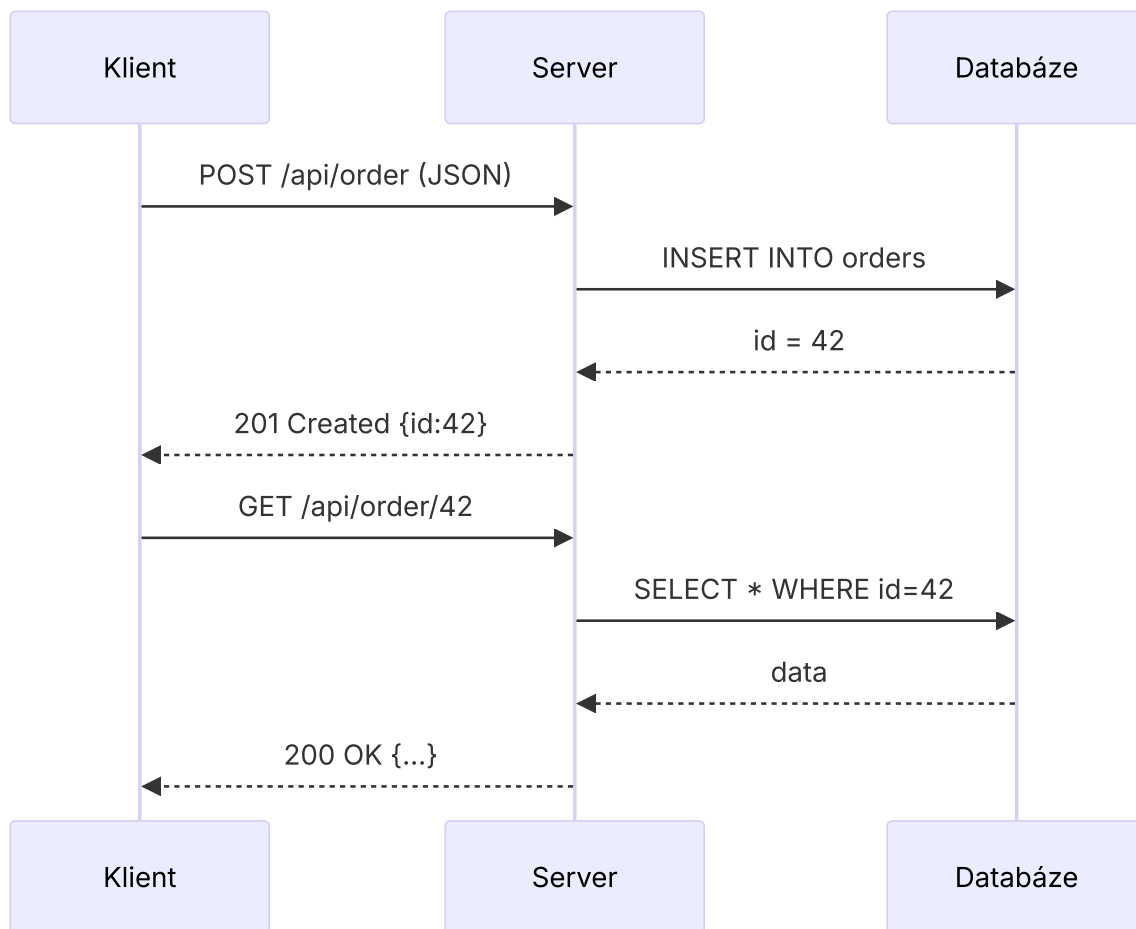
► ARCHITEKTURY

- **Klient-server** — frontend ↔ backend.
- **Třívrstvá** — prezentace (UI) → logika (API) → data (DB).
- **Mikroservisy** — backend rozdělen do menších služeb komunikujících přes HTTP/gRPC.
- **PWA** (Progressive Web App) — webová aplikace s vlastnostmi nativní (offline, install, push).

🔗 Praktický příklad

Internetový obchod: frontend je SPA v Reactu (na CDN), backend je ASP.NET REST API (aplikační server) a PostgreSQL. Při kliku „Přidat do košíku“ frontend pošle `POST /api/cart`, server odpoví `201 Created` s tělem (JSON). Pro SEO produktové stránky generuje Next.js (SSR) — Google dostane hotové HTML.

 Co nakreslit na potítku



20 Ověřování identity v prostředí internetu

💡 Elevator pitch

Autentizace ověřuje „kdo jsi“, autorizace „co smíš“. Na webu se prolínají hesla, tokeny, sociální přihlašování a vícefaktorové ověřování. Cílem je co nejméně otravovat uživatele a zároveň udržet účet bezpečný.

► HESLA

- Server nikdy neukládá heslo v plaintextu — **hash + sůl** (bcrypt, Argon2).
- Politika: minimální délka, kontrola proti slovníkům úniků (haveibeenpwned).
- **Brute force ochrana**: rate-limit, captcha, exponenciální zpoždění, dočasný lockout.
- Manažer hesel (Bitwarden, 1Password) — jedna master fráze.

► VÍCEFAKTOROVÉ OVĚŘENÍ (MFA / 2FA)

- Faktory: *vím* (heslo), *mám* (telefon, klíč), *jsem* (otisk, obličej).
- **TOTP** — Google Authenticator, Authy; 6-místný kód generovaný z klíče + času.
- **SMS** — pohodlné, ale zranitelné na SIM swap.
- **Push notifikace** v aplikaci.
- **FIDO2 / WebAuthn / passkeys** — hardwarový klíč nebo biometrie zařízení; bez hesla, phishing-resistant.

► SESSION A TOKENY

- **Session cookie** — server si pamatuje session v DB/cache, klient drží jen ID v cookie. Stateful.
- **Token (JWT)** — JSON Web Token, samonosný, podepsaný server (HMAC nebo RSA). Server nemusí držet stav.
- **Struktura JWT**: `header.payload.signature` (Base64URL).
 - Header: algoritmus, typ.
 - Payload: claims (sub, exp, iat, role...).
 - Signature: HMAC(header+payload, secret).
- **Bezpečné cookies**: `HttpOnly` (JS nečte), `Secure` (jen HTTPS), `SameSite=Lax/Strict` (CSRF ochrana).

► SOCIÁLNÍ PŘIHLAŠOVÁNÍ — OAUTH 2.0 A OPENID CONNECT

- **OAuth 2.0** — protokol pro *autorizaci*. Aplikace získá od uživatele souhlas a od poskytovatele *access token*, kterým volá API (Google, Facebook).
- **Authorization Code Flow + PKCE** — dnes doporučovaný pro webové i mobilní aplikace.

- **OpenID Connect (OIDC)** — vrstva nad OAuth 2.0 přidávající *autentizaci* přes `id_token` (JWT) — server tak ví, kdo přesně se přihlásil.
- Role: *Resource Owner* (uživatel), *Client* (naše aplikace), *Authorization Server* (Google), *Resource Server* (Google API).

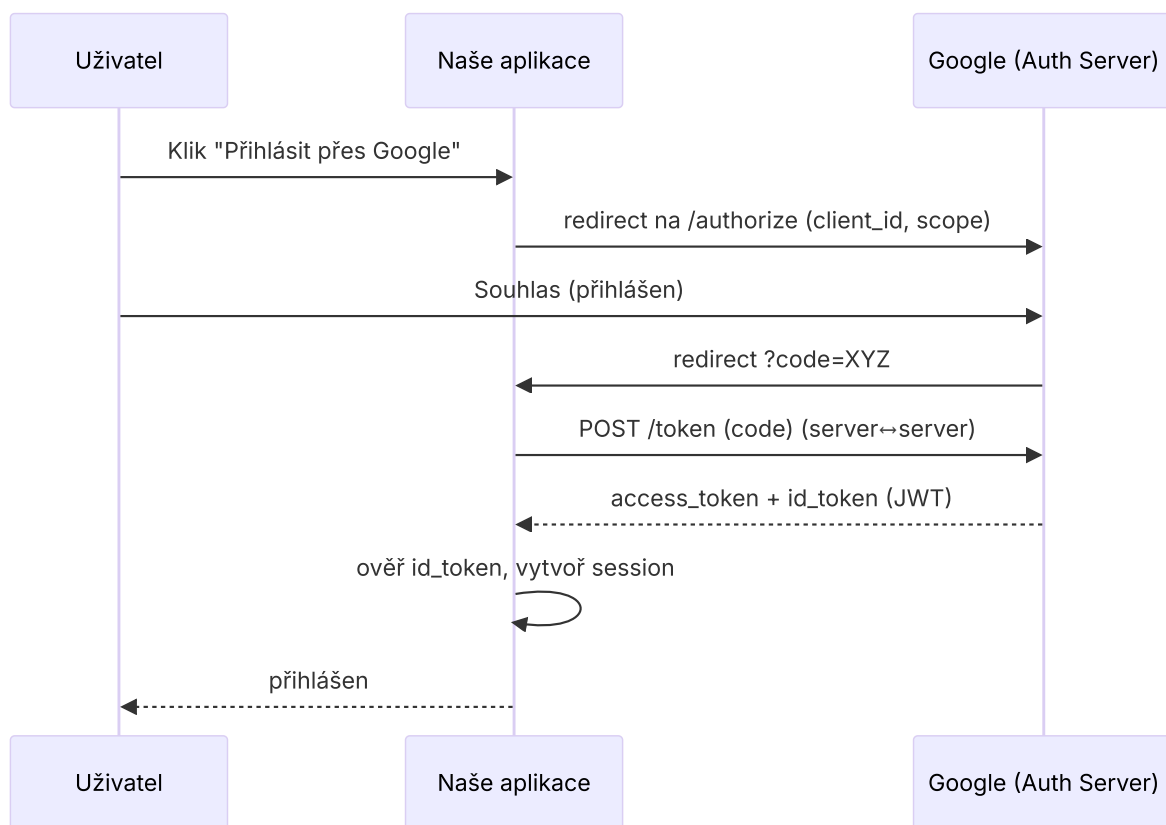
► ÚTOKY A OBRANA

Útok	Princip	Obrana
Phishing	Falešná stránka loví heslo	FIDO2/passkeys, kontrola URL
Brute force	Hádání hesel	Rate-limit, captcha, MFA
Credential stuffing	Uniklá hesla z jiných služeb	MFA, kontrola úniků
Session hijacking	Krádež cookie	HTTPS, HttpOnly, Secure, krátká životnost
CSRF	Podvržení požadavku z jiné stránky	SameSite cookie, CSRF token

Praktický příklad

V naší aplikaci nabídneme tři způsoby přihlášení: e-mail+heslo (s povinným TOTP po 1. úspěšném loginu), *Sign in with Google* (OIDC) a *passkey* přes WebAuthn. Heslo ukládáme přes Argon2id. Po loginu vydáme JWT s 15min platností + refresh token v HttpOnly cookie. Uživatel se nemusí přihlašovat každý den, ale ani my nemáme dlouhožitý zranitelný token.

 Co nakreslit na potítku



 Elevator pitch

REST (Representational State Transfer) je architektonický styl pro síťové API. Pracuje nad HTTP, modeluje data jako zdroje (resources) identifikované URL a operace nad nimi přes HTTP metody. RESTful API = API splňující principy REST.

► PRINCIPY REST (ROY FIELDING, 2000)

- **Klient-server** — oddělené odpovědnosti.
- **Bezstavovost** — každý request obsahuje vše potřebné, server si nedrží stav klienta.
- **Cachovatelnost** — odpovědi mají hlavičky o cache (Cache-Control , ETag).
- **Jednotné rozhraní (URI + HTTP metody + reprezentace).**
- **Vrstvená architektura** — klient nepozná, jestli mluví s API gateway, cache nebo backendem.
- **HATEOAS** (volitelné) — odpovědi obsahují odkazy na související akce.

► MAPA METOD ↔ OPERACE NAD ZDROJEM

Operace	Metoda	URL příklad	Stav. kód
Seznam	GET	/api/products	200
Detail	GET	/api/products/42	200 / 404
Vytvořit	POST	/api/products	201 Created + Location
Nahradit	PUT	/api/products/42	200 / 204
Upravit	PATCH	/api/products/42	200 / 204
Smazat	DELETE	/api/products/42	204 No Content

- URL pojmenováváme **podstatnými jmény v množném čísle** (/products , ne /getProducts).
- Vnořené zdroje: /users/7/orders/3 .
- Filtrování přes query: ?category=knihy&sort=cena .
- Stránkování: ?page=2&size=20 .
- Verzování: /api/v1/ ... nebo přes hlavičku.

► FORMÁTY DAT

	JSON	XML
Zápis	Klíč-hodnota, kratší	Tagy, plus atributy
Čitelnost	Lepší	Verbose
Validace	JSON Schema	XSD (vyspělá)
Použití	Web API, REST, JS-friendly	SOAP, finanční sektor, dokumenty

```
// JSON
{ "id": 42, "nazev": "Kniha", "cena": 299.00 }
```

```
<!-- XML -->
<produkt id="42">
  <nazev>Kniha</nazev>
  <cena>299.00</cena>
</produkt>
```

► AJAX A FETCH

- **AJAX** (Asynchronous JavaScript And XML) — historický termín pro asynchronní volání ze stránky bez přenačtení.
- Dnes `fetch()` nebo `axios` — vrací Promise.
- **CORS** (Cross-Origin Resource Sharing) — server musí povolit volání z jiného originu hlavičkou `Access-Control-Allow-Origin`.

```
const res = await fetch('/api/products', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json',
    'Authorization': 'Bearer ' + token },
  body: JSON.stringify({ nazev: 'Kniha', cena: 299 })
});
if (!res.ok) throw new Error(res.status);
const created = await res.json();
```

► STAVOVÉ KÓDY V REST

- Úspěch: 200, 201, 204.
- Klient: 400 (validace), 401 (chybí auth), 403 (nemá právo), 404, 409 (konflikt), 422 (semanticky neplatné).
- Server: 500, 502, 503.

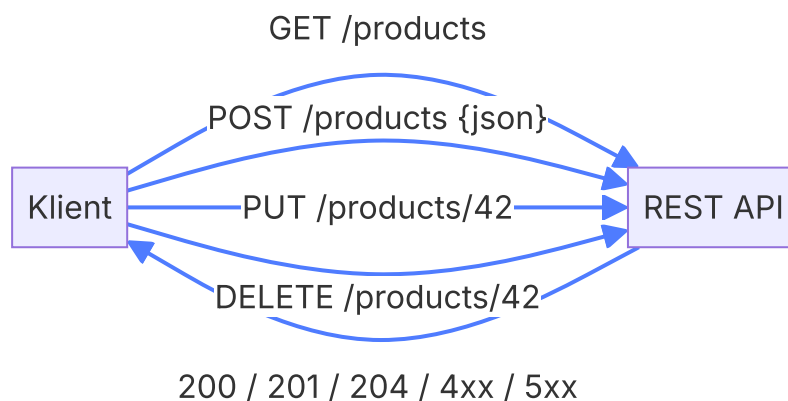
► REST VS. RPC VS. GRAPHQL

	REST	GraphQL	RPC (gRPC)
Endpointy	Mnoho zdrojů	Jediný (/graphql)	Volání metod
Výběr polí	Pevný	Klient určí	Pevný
Caching	HTTP nativně	Komplikovanější	Vlastní

✂ Praktický příklad

Pro mobilní aplikaci e-shopu navrhnu REST API: GET /api/v1/products?category=knihy&page=1 vrátí 200 OK + JSON pole. Vytvoření objednávky POST /api/v1/orders s JSON body, server odpoví 201 Created a hlavičkou Location: /api/v1/orders/42 . Při neoprávněném přístupu vrátí 401 , při nedostatku zboží 409 Conflict .

✏ Co nakreslit na potítku



💡 Elevator pitch

ASP.NET (Core) je multiplatformní framework od Microsoftu pro tvorbu webových aplikací a API v jazyce C#. Nabízí různé programovací modely — Razor Pages, MVC, Web API, Blazor — postavené nad jedním pipelinem požadavku.

► KLÍČOVÉ STAVEBNÍ KAMENY

- **Kestrel** — výchozí HTTP server, výkonný a multiplatformní.
- **Middleware pipeline** — řetěz komponent, kterým prochází každý request (auth, routing, statické soubory, atd.). Definuje se v `Program.cs`.
- **Dependency Injection (DI)** — vestavěný kontejner; služby se registrují (`builder.Services.AddScoped<IFooService, FooService>()`) a injektují konstruktorem.
- **Konfigurace** — `appsettings.json`, environment variables, secrets, čte se přes `IConfiguration / IOptions<T>`.
- **Hosting** — Generic Host, životní cyklus aplikace.

► PROGRAMOVACÍ MODEL Y

Model	Použití	Charakteristika
Razor Pages	Stránkově orientované weby	1 stránka = 1 <code>.cshtml</code> + <code>PageModel</code> ; jednoduché
MVC	Komplexnější weby	Controller → Action → View; striktní oddělení
Web API	REST/JSON služby	<code>ApiController</code> + atributové routy
Minimal API	Mikroslužby, jednoduché endpointy	<code>app.MapGet("/", () => "Hello");</code>
Blazor	SPA v C#	Server (SignalR) nebo WebAssembly

► RAZOR PAGES — JAK FUNGUJÍ

- Stránka má dva soubory: `Index.cshtml` (HTML+Razor) a `Index.cshtml.cs` (`PageModel` — třída s handlery).
- **Handlery**: `OnGet()`, `OnPost()`, případně `OnGetAsync()`, `OnPostNamed(...)` pro pojmenované akce (`asp-page-handler="Delete"`).
- **Bindování dat**: `[BindProperty] public InputModel Input { get; set; }` — automatická vazba formulářových polí na model.

- **Návratové metody:**

- Page() — znovu zobrazit stejnou stránku (typicky při validaci).
- RedirectToPage("Index") — PRG vzor (post-redirect-get).
- NotFound() , Content("...") , File(...) , Redirect(url) .

- **Routování** je file-based: Pages/Produkty/Detail.cshtml → /Produkty/Detail . Parametry přes @page "{id:int}" .

► **RAZOR SYNTAXE**

```
@* komentář *@
@page "{id:int?}"
@model IndexModel
<h1>Vítej, @Model.Jmeno</h1>

@if (Model.Jmeno != null) {
    <p>Hodnota je vyplněná.</p>
}

<ul>
    @foreach (var p in Model.Produkty) {
        <li>@p.Nazev – @p.Cena Kč</li>
    }
</ul>

<form method="post">
    <input asp-for="Input.Email" />
    <span asp-validation-for="Input.Email"></span>
    <button type="submit">odeslat</button>
</form>
```

► **TAG HELPERS A PRETTY URI**

- **Tag Helpers** — atributy asp-* generují HTML serverově (asp-for , asp-page , asp-route-id , asp-validation-for).
- **Pretty URI** — čisté, čitelné URL bez ?id=... , např. /produkty/42 dosažené přes @page "{id:int}" .

► **KONFIGURACE APLIKACE (PROGRAM.CS)**

```

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddRazorPages();
builder.Services.AddDbContext<AppDb>(o => o.UseSqlServer(
    builder.Configuration.GetConnectionString("Default")));
builder.Services.AddScoped<IEmailService, SmtplibEmailService>();

var app = builder.Build();

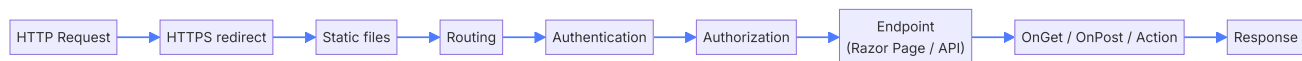
app.UseHttpsRedirection();
app.UseStaticFiles();
app.UseRouting();
app.UseAuthentication();
app.UseAuthorization();
app.MapRazorPages();
app.Run();

```

✂ Praktický příklad

Vytváříme registraci na kurz: `/Register` má `OnGet()` (zobrazí formulář, případně předvyplní e-mail z query) a `OnPost()` (validace, uložení, `RedirectToPage("Confirm")`). `InputModel` s `[Required]` a `[EmailAddress]` atributy zajistí validaci. Tag Helpery `asp-for` + `asp-validation-for` automaticky propojí formulář s modelem a chybové hlášky.

✂ Co nakreslit na potítku



23 Událostmi řízené programování

💡 Elevator pitch

Místo aby program běžel od shora dolů, čeká na události (kliknutí, příchod zprávy, časovač) a reaguje na ně. Klíčový je vzor Publisher–Subscriber, ve kterém zdroj události oznamuje její vznik všem zájemcům.

► PRINCIP A HLAVNÍ POJMY

- **Událost (Event)** — něco, co nastalo (klik, doručení dat).
- **Publisher (vydavatel, zdroj)** — kdo událost vyhláší.
- **Subscriber (odběratel, handler)** — kdo se k události přihlásil a reaguje.
- **Event loop** — smyčka, která bere události z fronty a předává handlerům (UI vlákno, JavaScript, Node).
- **Callback** — funkce předaná jako parametr, která se zavolá později.
- **Delegate** (C#) — typově bezpečný odkaz na metodu; základ `event` .

► SOUVISEJÍCÍ NÁVRHOVÉ VZORY

- **Observer** — publisher drží seznam subscriberů a iterativně je notifikuje.
- **Mediator** — centrální prostředník, místo aby publisheri znali subscribery napřímo.
- **Command** — událost je objekt s metodou `Execute()` .
- **Pub/Sub bus** — message broker (RabbitMQ, Kafka) — distribuovaný Observer.

► PŘIHLÁŠENÍ A ODHLÁŠENÍ (C#)

```
public class Tlacitko {  
    public event EventHandler Kliknuto;           // delegate  
  
    public void Stiskni() =>  
        Kliknuto?.Invoke(this, EventArgs.Empty); // ?. = je-li někdo přihlášený  
}
```

```
var btn = new Tlacitko();  
EventHandler handler = (s, e) => Console.WriteLine("Klik!");  
btn.Kliknuto += handler; // přihlášení (subscribe)  
btn.Stiskni();           // → "Klik!"  
btn.Kliknuto -= handler; // odhlášení (unsubscribe)
```

- **Memory leak** — pokud subscriber zapomene `-=` , publisher na něj drží referenci a GC ho nesmaže. Řešení: weak references, slabé eventy, `IDisposable` .
- V JavaScriptu: `btn.addEventListener('click', fn) / removeEventListener` .

► SYNCHRONNÍ VS. ASYNCHRONNÍ UDÁLOST

	Synchronní	Asynchronní
Volání	Publisher čeká na všechny handlers	Handler běží na pozadí (Task, vlákno, message bus)
Pořadí	Deterministické	Nezaručené
Vhodné pro	UI, kde reakce musí být okamžitá	Velký objem zpráv, distribuované systémy

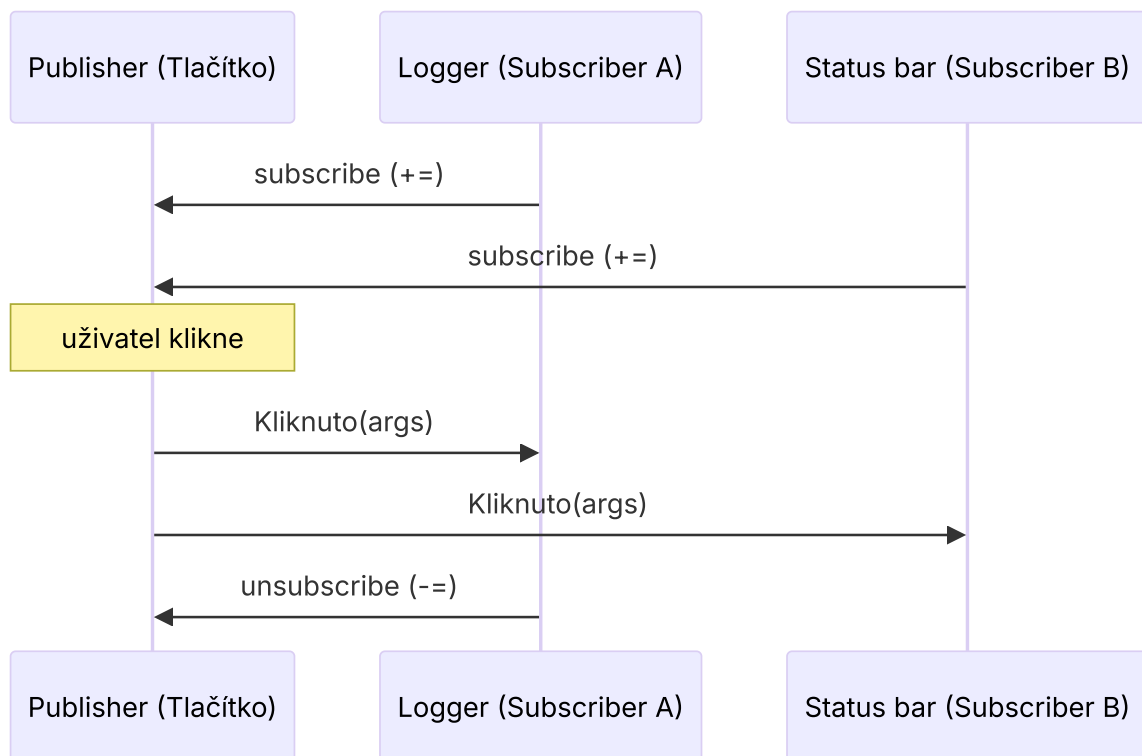
► PULL VS. PUSH MODEL

- **Pull** — kód se opakovaně dotazuje (`while (input.Available) ...`). Spotřebovává CPU.
- **Push (event-driven)** — systém zavolá handler, když má co předat. Efektivnější.

✂ Praktický příklad

V chatovací aplikaci publishera představuje SignalR hub. Když pošlu zprávu, hub vyhlásí událost `NovaZprava`, na kterou jsou přihlášení všichni klienti místnosti. Žádný klient se neptá „*máš pro mě něco?*“ — jen čeká, až mu server pošle událost. UI komponenta v Reactu má `useEffect`, který v `cleanu` odhlásí handler — předejde leakům, když uživatel místnost opustí.

✏ Co nakreslit na potítku



 Elevator pitch

Programovací jazyk je formální nástroj, kterým zapisujeme algoritmus tak, aby ho počítač uměl provést. Jazyky se liší úrovní abstrakce (assembler vs. C#), paradigmatem (imperativní, OOP, funkcionální) a způsobem překladu.

► ÚROVEŇ ABSTRAKCE

- **Strojový kód** — binární instrukce konkrétního procesoru (x86, ARM).
- **Assembler** — symbolický zápis instrukcí 1:1 (MOV, ADD).
- **Nízkoúrovňové** — C, C++ (manuální správa paměti, blízko HW).
- **Vysokoúrovňové** — Java, C#, Python (automatická paměť, bohatá knihovna).
- **4GL/DSL** — SQL, HTML, regex; specializované na doménu.

► PARADIGMATA

Paradigma	Idea	Příklady
Imperativní / procedurální	Sled příkazů, jak něco udělat	C, Pascal
Objektově orientované	Objekty se stavem a chováním	C#, Java, C++, Python
Funkcionální	Funkce jako hodnota, neměnnost	Haskell, F#, Elixir, JS (částečně)
Logické	Fakta + pravidla → odvození	Prolog
Skriptovací	Rychlé psaní, dynamické	Python, JS, Bash

► PŘEKLAD JAZYKA DO STROJOVÉHO KÓDU

- **Kompilátor (compiler)** — celý zdroj přeloží před spuštěním do strojového kódu (.exe). Rychlý běh. (C, C++, Go, Rust)
- **Interpret** — čte a vykonává řádek po řádku za běhu. (Python, klasický JS)
- **Hybrid (mezikód)** — překlad do *bytecode*, ten běží na virtuálním stroji a často se za běhu JIT-kompiluje do nativního kódu. (.NET CIL/IL → CLR; Java bytecode → JVM)
- **Transpilace** — překlad do jiného jazyka stejné úrovně (TypeScript → JS, Babel JSX → JS).

```
// Fáze překladu (zjednodušeně)
1. Lexikální analýza - tokenizace
2. Syntaktická analýza - AST
3. Sémantická analýza - kontrola typů
4. Optimalizace
5. Generování kódu (mezikódu / nativního)
```

► MULTIPLATFORMNOST

- **Recompile** — stejný zdroj, různé cíle (C/C++ pro Windows, Linux).
- **Mezikód + VM** — Java JAR běží všude, kde je JVM; .NET assembly v CLR.
- **WebAssembly** — přenosný bytecode pro web (běží i mimo prohlížeč přes WASI).

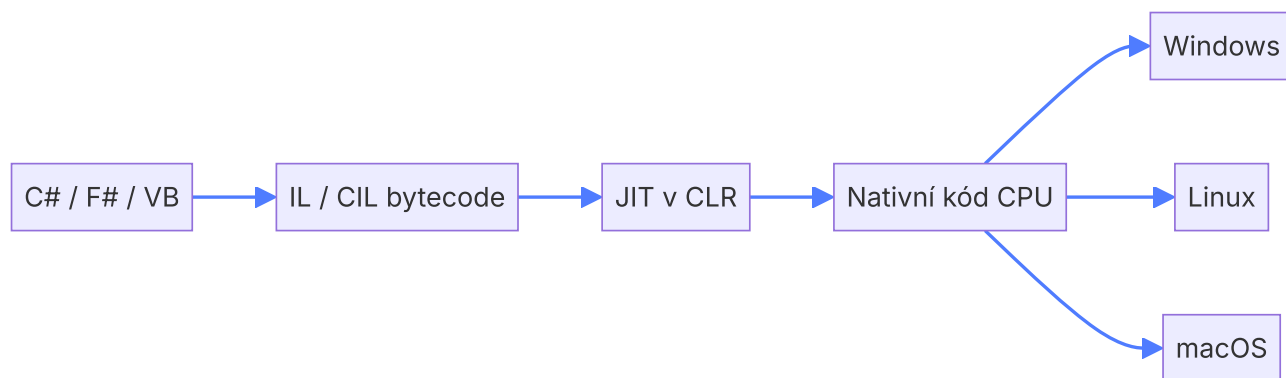
► ARCHITEKTURA .NET

- **.NET** (dříve .NET Core) — multiplatformní open-source platforma od Microsoftu (Win/Linux/macOS).
- **CLR** (Common Language Runtime) — virtuální stroj; spravuje paměť (GC), spouští kód.
- **CIL / IL** (Common Intermediate Language) — mezikód, do kterého se překládají všechny .NET jazyky.
- **JIT** (Just-In-Time) kompilátor — IL → nativní kód za běhu. Případně **AOT** (Ahead-Of-Time) pro startup.
- **BCL** (Base Class Library) — sdílené knihovny (System.IO, System.Collections...).
- **Jazyky**: C#, F# (funkcionální), VB.NET — všechny se kompilují do stejného IL → CLI komponenty mohou volat jedna druhou.
- **NuGet** — správce balíčků.
- **CTS / CLS** — Common Type System a Common Language Specification — zajišťují interoperabilitu jazyků.

✂ Praktický příklad

Tým má backend v C#, datovou vědu v F# a web frontend v TypeScriptu. Backend i F# kód se kompiluje do **IL** → běží v CLR na Linux serveru bez závislosti na Windows. TypeScript se transpiluje do JavaScriptu, který běží v prohlížeči. Jeden vývojář může bez změny zdroje aplikaci pustit lokálně na Macu i v Dockeru na Linuxu — to je síla mezikódu a multiplatformnosti.

 Co nakreslit na potítku



💡 Elevator pitch

Android je vrstvený OS postavený na Linuxu. Aplikace se skládají ze čtyř hlavních komponent (Activity, Service, BroadcastReceiver, ContentProvider), běží v izolovaném sandboxu a komunikují přes Intenty. Moderní aplikace staví UI v Jetpack Compose a logiku v MVVM.

► VRSTVY ANDROID OS (ZDOLA NAHORU)

- **Linux Kernel** — ovladače, paměť, procesy, energetika, bezpečnost.
- **HAL** (Hardware Abstraction Layer) — sjednocené rozhraní k senzorům, kameře.
- **Native libs + Android Runtime (ART)** — VM pro běh aplikací; používá AOT i JIT kompilaci, garbage collector. Předchůdce Dalvik.
- **Java/Kotlin API Framework** — Activity Manager, Window Manager, Notifications.
- **System apps + Apps** — vlastní aplikace.

► SANDBOX A BEZPEČNOST

- Každá aplikace má vlastní UID a běží ve svém procesu.
- Nemá přístup k souborům jiných aplikací (jen pokud je explicitně sdílí).
- **Oprávnění** — install-time (běžné) vs. runtime (nebezpečné — kamera, poloha) — uživatel je schvaluje za běhu.

► ANDROIDMANIFEST.XML

- Centrální popis aplikace: package, verze, ikona, požadovaná oprávnění (`<uses-permission>`), seznam komponent (`<activity>` , `<service>` , `<receiver>` , `<provider>`) a jejich Intent filtry.

► ČTYŘI HLAVNÍ KOMPONENTY

Komponenta	Účel	Příklad
Activity	Jedna obrazovka s UI	Login, Detail, Seznam
Service	Dlouhotrvající operace bez UI	Přehrávač hudby, sync
Broadcast Receiver	Posluchač systémových/vlastních broadcastů	Změna baterie, příchozí SMS
Content Provider	Sdílení dat mezi aplikacemi	Kontakty, kalendář

► INTENT

- „Záměr“ — zpráva pro jinou komponentu.
- **Explicitní** — určí konkrétní cílovou třídu (`Intent(this, Detail::class.java)`).
- **Implicitní** — popíše akci (`ACTION_VIEW` , `ACTION_SEND`) — systém najde aplikaci, která ji umí zpracovat (sdílení, otevření URL).
- Mohou nést data (extras) a flagy.

► LIFECYCLE ACTIVITY

```
onCreate → onStart → onResume → (běží)
                        ↓ uživatel skryje
onPause → onStop → onDestroy
                        ↑ vrátí se
onRestart → onStart → onResume
```

- **onCreate** — inflate UI, inicializace.
- **onResume** — aktivita je viditelná a v popředí.
- **onPause** — uložit stav, zastavit animace; následuje `onStop` , pokud aktivita zmizí.
- Lifecycle Components (Jetpack) — `LifecycleOwner` , `LiveData` , `StateFlow` — UI reaguje na stav.

► JETPACK COMPOSE

- Moderní deklarativní UI toolkit pro Android (od 2021).
- Místo XML layoutů píšeme **composable funkce** (`@Composable fun Greet() { Text("Ahoj") }`).
- **State hoisting** — stav se drží výš a předává dolů, podobně jako v Reactu.
- Plynulá integrace s Kotlin coroutines, Flow.

► MVVM (MODEL–VIEW–VIEWMODEL)

- **Model** — data, repozitáře (DB, API).
- **View** — Composable funkce / Activity; jen zobrazuje stav.
- **ViewModel** — drží UI stav (`StateFlow`), zpracovává akce, přežívá rotaci obrazovky (Jetpack `ViewModel`).
- **Data binding** — View pozoruje `StateFlow` / `LiveData` a překresluje se.

```

class CounterViewModel : ViewModel() {
    private val _count = MutableStateFlow(0)
    val count: StateFlow<Int> = _count
    fun increment() { _count.value++ }
}

@Composable
fun CounterScreen(vm: CounterViewModel = viewModel()) {
    val c by vm.count.collectAsState()
    Column {
        Text("Hodnota: $c")
        Button(onClick = { vm.increment() }) { Text("+1") }
    }
}

```

✂ Praktický příklad

Aplikace pro nákupní seznam: **Activity** hostuje Compose obsah. **ViewModel** drží `StateFlow<List<Item>>`, načítá data z **Room** (lokální DB) přes repozitář (Model). **Service** na pozadí synchronizuje s cloudem, **BroadcastReceiver** reaguje na `B00T_COMPLETED` a naplánuje sync. Sdílení seznamu řešíme implicitním Intentem `ACTION_SEND`. Manifest deklaruje oprávnění `INTERNET` a `POST_NOTIFICATIONS`.

✂ Co nakreslit na potítku

